

Computer Architecture

CS 622 / EE 520

Spring 2026

Lecture 4
Shahid Masud

Frequency of FP operations = 25%

Average CPI of FP operations = 4.0

Average CPI of other instructions = 1.33

Frequency of FSQRT = 2%

CPI of FSQRT = 20

From last lecture

There are two possibilities for improvement:

1. Improve all FP operations
2. Improve FSQRT operation

Assume that the two design alternatives are to decrease the CPI of FSQRT to 2 or to decrease the average CPI of all FP operations to 2.5. Compare these two design alternatives using the processor performance equation.

Answer First, observe that only the CPI changes; the clock rate and instruction count remain identical. We start by finding the original CPI with neither enhancement:

$$\begin{aligned}\text{CPI}_{\text{original}} &= \sum_{i=1}^n \text{CPI}_i \times \left(\frac{\text{IC}_i}{\text{Instruction count}} \right) \\ &= (4 \times 25\%) + (1.33 \times 75\%) = 2.0\end{aligned}$$

We can compute the CPI for the enhanced FSQRT by subtracting the cycles saved from the original CPI:

$$\begin{aligned}\text{CPI}_{\text{with new FSQRT}} &= \text{CPI}_{\text{original}} - 2\% \times (\text{CPI}_{\text{old FSQRT}} - \text{CPI}_{\text{of new FSQRT only}}) \\ &= 2.0 - 2\% \times (20 - 2) = 1.64\end{aligned}$$

We can compute the CPI for the enhancement of all FP instructions the same way or by summing the FP and non-FP CPIs. Using the latter gives us

$$\text{CPI}_{\text{new FP}} = (75\% \times 1.33) + (25\% \times 2.5) = 1.625$$

Since the CPI of the overall FP enhancement is slightly lower, its performance will be marginally better. Specifically, the speedup for the overall FP enhancement is

$$\begin{aligned}\text{Speedup}_{\text{new FP}} &= \frac{\text{CPU time}_{\text{original}}}{\text{CPU time}_{\text{new FP}}} = \frac{\text{IC} \times \text{Clock cycle} \times \text{CPI}_{\text{original}}}{\text{IC} \times \text{Clock cycle} \times \text{CPI}_{\text{new FP}}} \\ &= \frac{\text{CPI}_{\text{original}}}{\text{CPI}_{\text{new FP}}} = \frac{2.00}{1.625} = 1.23\end{aligned}$$

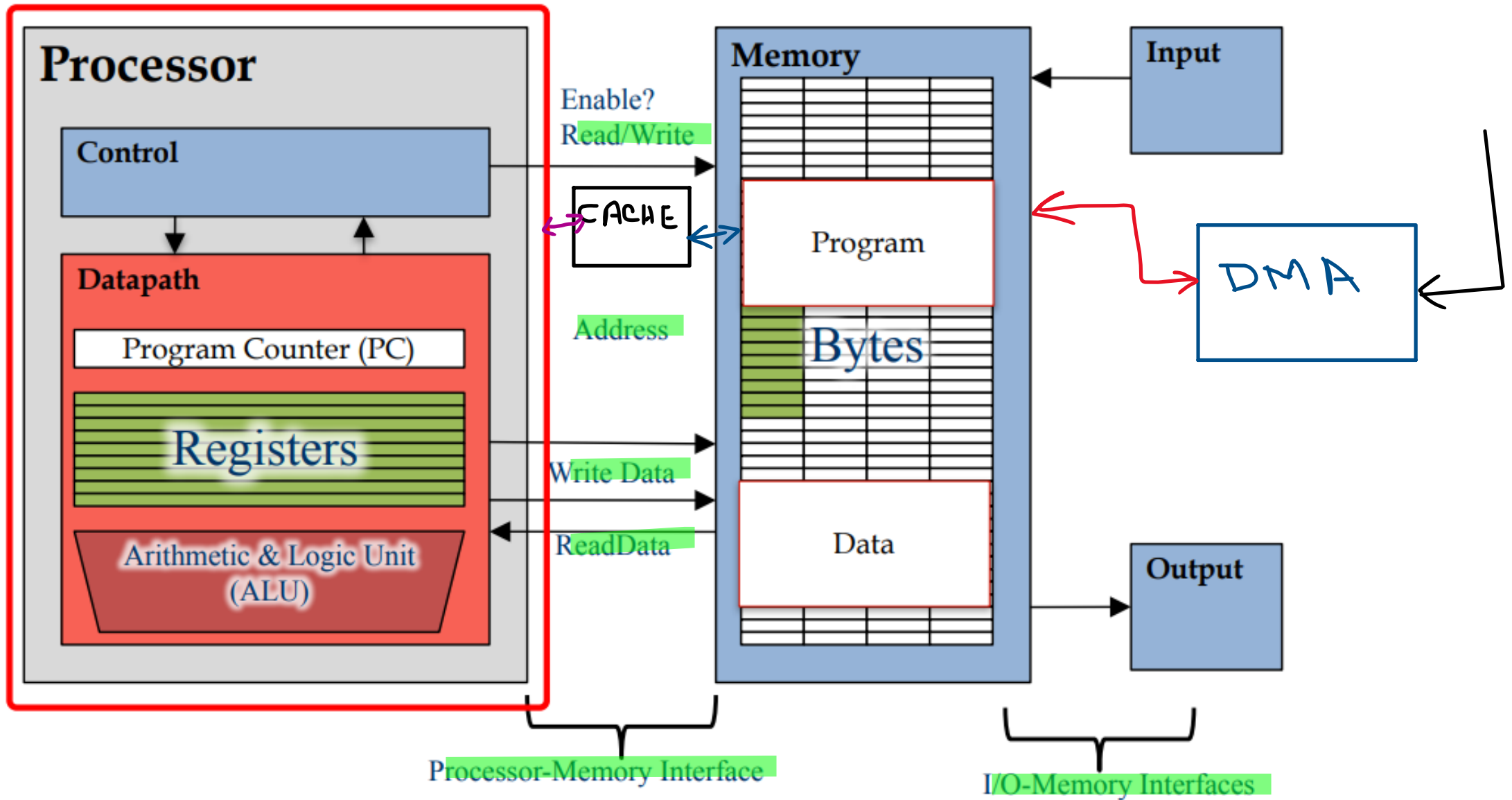
- Development of Instruction Set Architecture
 - Classifying Instruction Set Architectures
 - Memory Access
 - Role of CPU Registers in Architectures
 - Data Types and Size of Operands
 - Memory Addressing and Alignment
 - Operations in the Instruction Set
 - Instructions for Control Flow
 - The Role of Compilers
 - Introducing RISC-V ISA

Quiz 1
Next Lecture

RISC-V ISA from Appendix A

Instruction Set Architecture (ISA)

Components of a Computer



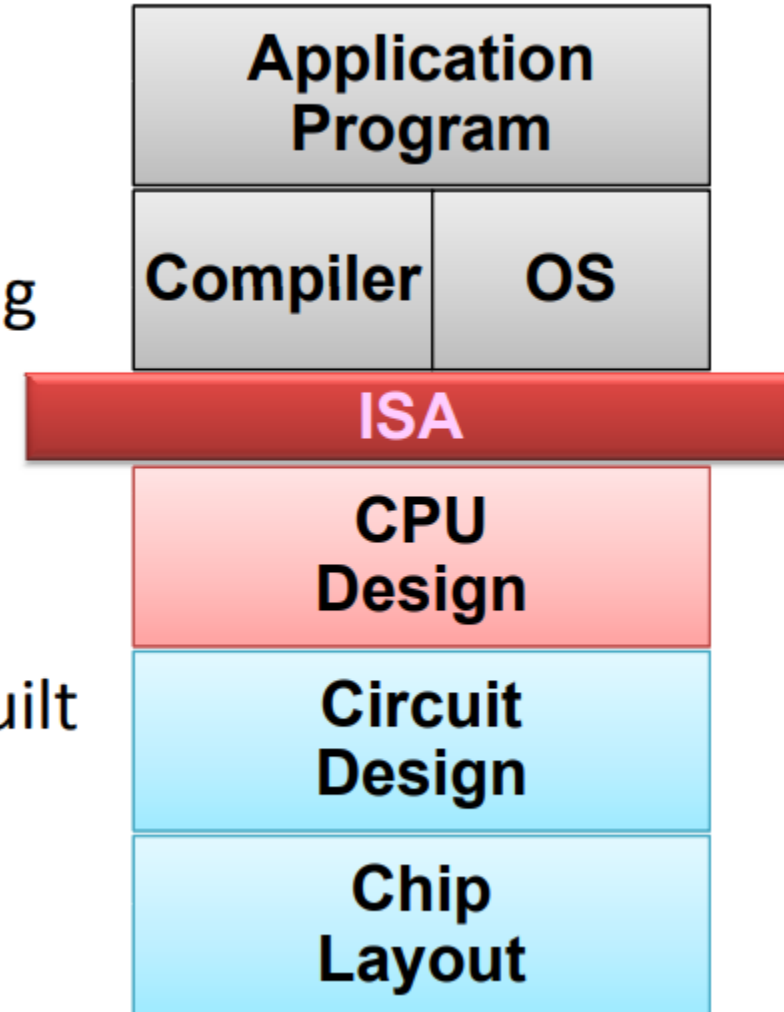
What is an Instruction Set Architecture?



- Lowest level of Computer Architecture that is visible to a programmer
- ISA comprises instructions in Assembly Language and Corresponding Binary Machine Code
- Primitive syntax compared to a high-level language
- It is easily interpreted and understood by the CPU
- Designed to maximize performance
- Represents features and performance of a CPU
- Designed to minimize cost and design time

Role of ISA within a Computer

- Assembly Language View
 - Processor state (RF, mem)
 - Instruction set and encoding
- Layer of Abstraction
 - Above: how to program machine - HLL, OS
 - Below: what needs to be built - tricks to make it run fast



Key:

RF = Register File

Mem = Memory

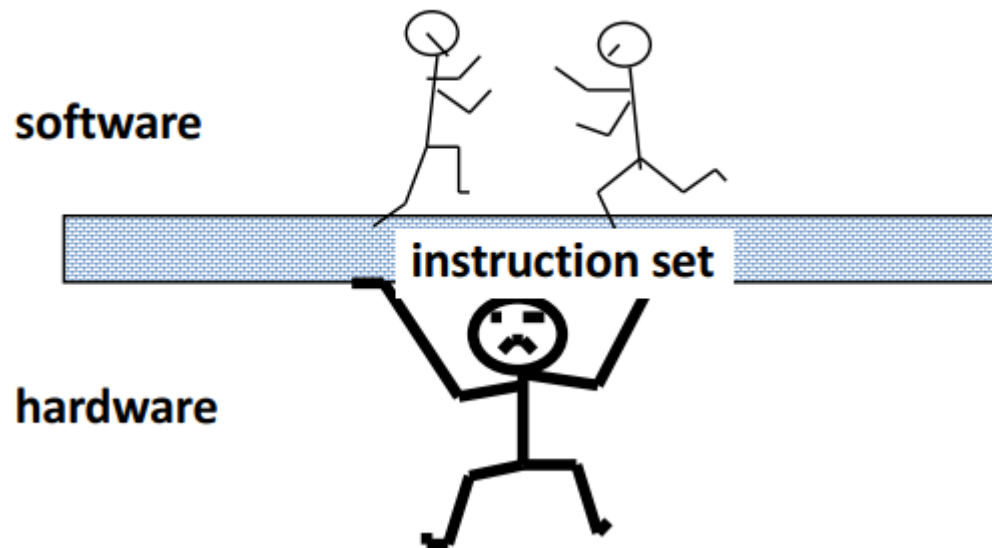
HLL = High Level Language

OS = Operating System

Instruction Set Architecture

Instruction Set Architecture – the portion of the machine visible to the assembly level programmer or to the compiler writer

- To use the hardware of a computer, we must speak its language
- The words of a computer language are called instructions, and its vocabulary is called an instruction set



Instr. #	Operation+Operands
<i>i</i>	movl -4(%ebp), %eax
<i>(i+1)</i>	addl %eax, (%edx)
<i>(i+2)</i>	cmpl 8(%ebp), %eax
<i>(i+3)</i>	j1 L5
:	
L5:	

Instruction Set Architecture: The Myopic View of Computer Architecture

We use the term *instruction set architecture* (ISA) to refer to the actual programmer-visible instruction set in this book. The ISA serves as the boundary between the software and hardware. This quick review of ISA will use examples from 80x86, ARMv8, and RISC-V to illustrate the seven dimensions of an ISA. The most popular RISC processors come from ARM (Advanced RISC Machine), which were in 14.8 billion chips shipped in 2015, or roughly 50 times as many chips that shipped with 80x86 processors. Appendices A and K give more details on the three ISAs.

RISC-V (“RISC Five”) is a modern RISC instruction set developed at the University of California, Berkeley, which was made free and openly adoptable in response to requests from industry. In addition to a full software stack (compilers, operating systems, and simulators), there are several RISC-V implementations freely available for use in custom chips or in field-programmable gate arrays. Developed 30 years after the first RISC instruction sets, RISC-V inherits its ancestors’ good ideas—a large set of registers, easy-to-pipeline instructions, and a lean set of operations—while avoiding their omissions or mistakes. It is a free and open, elegant example of the RISC architectures mentioned earlier, which is why more than 60 companies have joined the RISC-V foundation, including AMD, Google, HP Enterprise, IBM, Microsoft, Nvidia, Qualcomm, Samsung, and Western Digital. We use the integer core ISA of RISC-V as the example ISA in this book.

- Generally kept very simple and extendable
- Separated into multiple specifications
 - User-Level ISA spec (compute instructions)
 - Compressed ISA spec (16-bit instructions)
 - Privileged ISA spec (supervisor-mode instructions)
 - More ...
- ISA support is given by RV + word-width + extensions supported
 - E.g. RV32I means 32-bit RISC-V with support for the I(nteger) instruction set

- Defines the normal instructions needed for computation
 - A mandatory **Base integer ISA**
 - **I: Integer instructions:**
 - ALU
 - Branches/jumps
 - Loads/stores
 - **Standard Extensions**
 - **M: Integer Multiplication and Division**
 - **A: Atomic Instructions**
 - **F: Single-Precision Floating-Point**
 - **D: Double-Precision Floating-Point**
 - **C: Compressed Instructions (16 bit)**
 - **G = IMAFD: Integer base + four standard extensions**
 - Optional extensions

RISC-V Variants

- Both 32-bit and 64-bit address space variants
 - RV32 and RV64
- Easy to subset/extend for education/research
 - RV32IM, RV32IMA, RV32IMAFD, RV32G
- SPEC on the website
 - www.riscv.org

Name of base or extension	Functionality
RV32I	Base 32-bit integer instruction set with 32 registers
RV32E	Base 32-bit instruction set but with only 16 registers; intended for very low-end embedded applications
RV64I	Base 64-bit instruction set; all registers are 64-bits, and instructions to move 64-bit from/to the registers (LD and SD) are added
M	Adds integer multiply and divide instructions
A	Adds atomic instructions needed for concurrent processing; see Chapter 5
F	Adds single precision (32-bit) IEEE floating point, includes 32 32-bit floating point registers, instructions to load and store those registers and operate on them
D	Extends floating point to double precision, 64-bit, making the registers 64-bits, adding instructions to load, store, and operate on the registers
Q	Further extends floating point to add support for quad precision, adding 128-bit operations
L	Adds support for 64- and 128-bit decimal floating point for the IEEE standard
C	Defines a compressed version of the instruction set intended for small-memory-sized embedded applications. Defines 16-bit versions of common RV32I instructions
V	A future extension to support vector operations (see Chapter 4)
B	A future extension to support operations on bit fields
T	A future extension to support transactional memory
P	An extension to support packed SIMD instructions: see Chapter 4
RV128I	A future base instruction set providing a 128-bit address space

- **Data Alignment**
 - 32-bit bus vs 8-bit memory
 - Big Endian vs Little Endian
- **Addressing Modes**
- **Type and Size of Operands**
 - I, R types
- **Operations available**
 - Arithmetic
 - Logical
 - Shift
- **Control Flow Instructions**
 - Branch
 - Jump

Memory Alignment

CPU Registers

Memory Access – Big Endian vs Little Endian

A CPU Register **R0** is 32-bits wide.

It has data “1A2B3C4D” Hex.

Store the data in an 8-bit wide RAM that has 4 locations.

Store in **Big-Endian style**

or Store in Little-Endian Style

CPU Register
R0 Contents

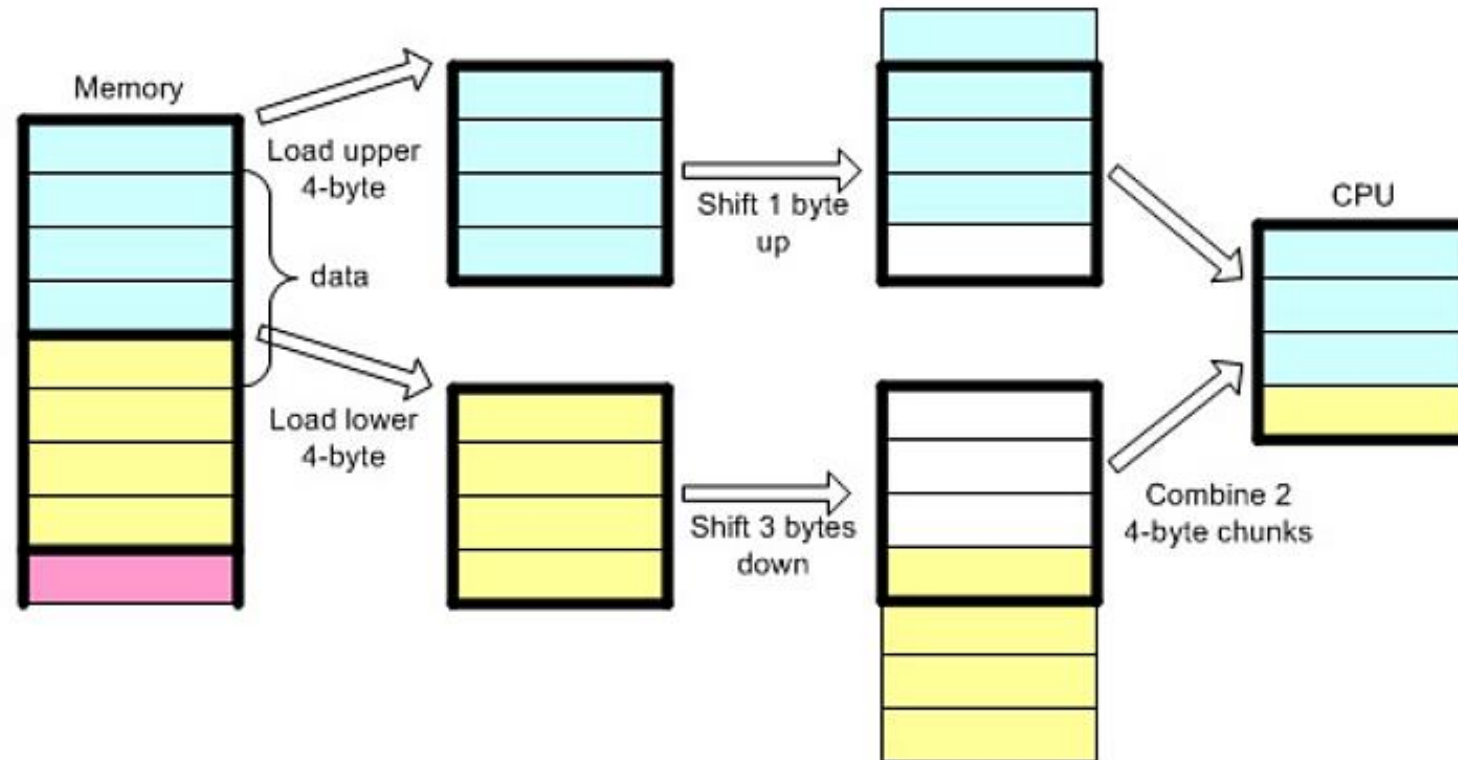
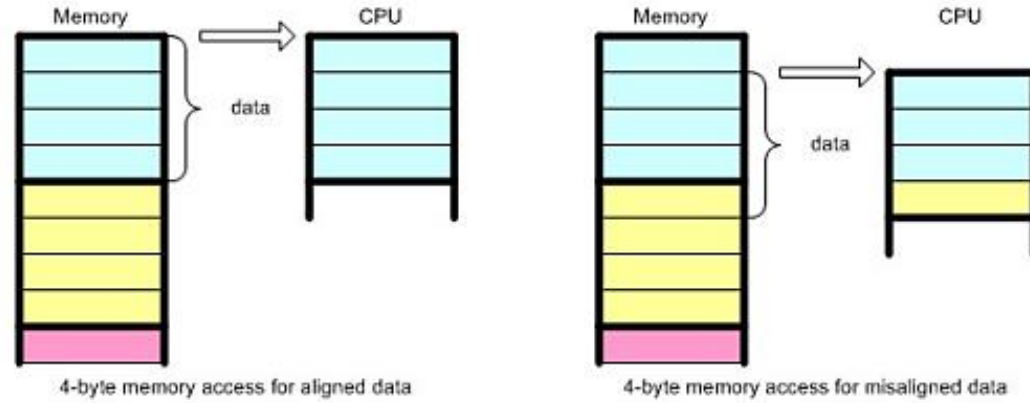


32-bit Register

Location	Big Endian RAM
3	
2	Byte width
1	
0	

Location	Little Endian RAM
3	
2	Byte Width
1	
0	

Memory Alignment



RISC-V Registers Naming Conventions

Integer

F.P.

Register	Name	Use	Saver
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Function arguments/return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-f7	ft0-ft7	FP temporaries	Caller
f8-f9	fs0-fs1	FP saved registers	Callee
f10-f11	fa0-fa1	FP function arguments/return values	Caller
f12-f17	fa2-fa7	FP function arguments	Caller
f18-f27	fs2-fs11	FP saved registers	Callee
f28-f31	ft8-ft11	FP temporaries	Caller

Figure 1.4 RISC-V registers, names, usage, and calling conventions. In addition to the 32 general-purpose registers (x0–x31), RISC-V has 32 floating-point registers (f0–f31) that can hold either a 32-bit single-precision number or a 64-bit double-precision number. The registers that are preserved across a procedure call are labeled “Callee” saved.

RISC-V is a Register-rich Architecture

These Registers are Inside the CPU and help in data processing And Storage of Results from ALU

Instruction Fields and Types

RISC-V Instruction Formats and Types

32-bit wide Instructions

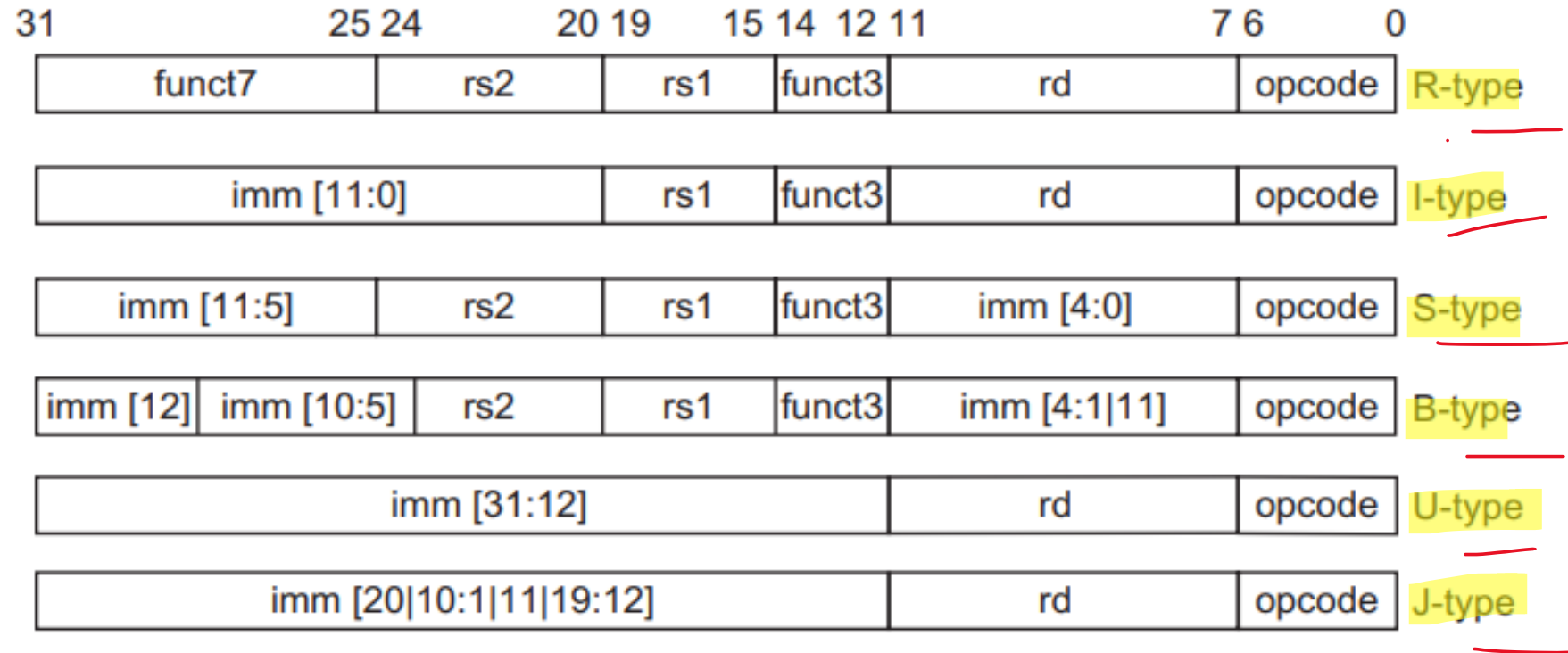


Figure 1.7 The base RISC-V instruction set architecture formats. All instructions are 32 bits long. The R format is for integer register-to-register operations, such as ADD, SUB, and so on. The I format is for loads and immediate operations, such as LD and ADDI. The B format is for branches and the J format is for jumps and link. The S format is for stores. Having a separate format for stores allows the three register specifiers (rd, rs1, rs2) to always be in the same location in all formats. The U format is for the wide immediate instructions (LUI, AUIPC).

RISC-V Encoding Summary



Name (Field Size)	Field						Comments
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

Use of Instruction Fields

Instruction format	Primary use	rd	rs1	rs2	Immediate
R-type	Register-register ALU instructions	Destination	First source	Second source	
I-type	ALU immediates Load	Destination	First source base register		Value displacement
S-type	Store Compare and branch		Base register first source	Data source to store second source	Displacement offset
U-type	Jump and link Jump and link register	Register destination for return PC	Target address for jump and link register		Target address for jump and link

Figure A.24 The use of instruction fields for each instruction type. Primary use shows the major instructions that use the format. A blank indicates that the corresponding field is not present in this instruction type. The I-format is used for both loads and ALU immediates, with the 12-bit immediate holding either the value for an immediate or the displacement for a load. Similarly, the S-format encodes both store instructions (where the first source register is the base register and the second contains the register source for the value to store) and compare and branch instructions (where the register fields contain the sources to compare and the immediate field specifies the offset of the branch target). There are actually two other formats: SB and UJ that follow the same basic organization as S and J, but slightly modify the interpretation of the immediate fields.

RISC-V Data Type Instructions

Instruction type/opcode	Instruction meaning
<i>Data transfers</i>	<i>Move data between registers and memory, or between the integer and FP or special registers; only memory address mode is 12-bit displacement+contents of a GPR</i>
lb, lbu, sb	Load <u>byte</u> , load byte unsigned, store byte (to/from integer registers)
lh, lhu, sh	Load <u>half word</u> , load half word unsigned, store half word (to/from integer registers)
lw, lwu, sw	Load <u>word</u> , load word unsigned, store word (to/from integer registers)
ld, sd	Load <u>double word</u> , store double word (to/from integer registers)
flw, fld, fsw, fsd	Load <u>SP float</u> , load <u>DP float</u> , store <u>SP float</u> , store <u>DP float</u>
fmv.?.x, fmv.x.?	Copy <u>from/to integer register to/from floating-point register</u> ; “?” = S for single-precision, D for double-precision
csrrw, csrrwi, csrrs, csrrsi, csrrc, csrrci	Read counters and write status registers, which include counters: clock cycles, time, instructions retired

RISC-V Arithmetic / Logical Instructions

Arithmetic/logical

add, addi, addw, addiw

sub, subw

mul, mulw, mulh, mulhsu,
mulhu

div, divu, rem, remu

divw, divuw, remw, remuw

and, andi

or, ori, xor, xori

lui

auipc

sll, slli, srl, srli, sra,
srai

sllw, slliw, srlw, srliw,
sraw, sraiw

slt, slti, sltu, sltiu

Operations on integer or logical data in GPRs

Add, add immediate (all immediates are 12 bits), add 32-bits only & sign-extend to 64 bits, add immediate 32-bits only

Subtract, subtract 32-bits only

Multiply, multiply 32-bits only, multiply upper half, multiply upper half signed-unsigned, multiply upper half unsigned

Divide, divide unsigned, remainder, remainder unsigned

Divide and remainder: as previously, but divide only lower 32-bits, producing 32-bit sign-extended result

And, and immediate

Or, or immediate, exclusive or, exclusive or immediate

Load upper immediate; loads bits 31-12 of register with immediate, then sign-extends

Adds immediate in bits 31-12 with zeros in lower bits to PC; used with JALR to transfer control to any 32-bit address

Shifts: shift left logical, right logical, right arithmetic; both variable and immediate forms

Shifts: as previously, but shift lower 32-bits, producing 32-bit sign-extended result

Set less than, set less than immediate, signed and unsigned

Control

beq, bne, blt, bge, bltu,
bgeu

Conditional branches and jumps; PC-relative or through register

Branch GPR equal/not equal; less than; greater than or equal, signed and unsigned

jal, jalr

Jump and link: save PC+4, target is PC-relative (JAL) or a register (JALR); if specify x0 as destination register, then acts as a simple jump

ecall

Make a request to the supporting execution environment, which is usually an OS

ebreak

Debuggers used to cause control to be transferred back to a debugging environment

fence, fence.i

Synchronize threads to guarantee ordering of memory accesses; synchronize instructions and data for stores to instruction memory

Classifying ISA

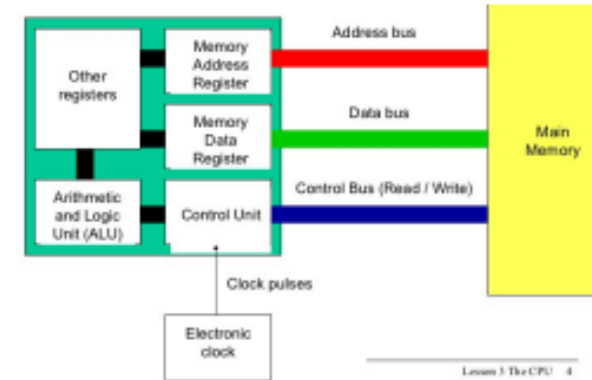
Classifying Instruction Set Architectures



Operand storage in CPU	Where are they other than memory
# explicit operands named per instruction	How many? Min, Max, Average
Addressing mode	How the effective address for an operand calculated? Can all use any mode?
Operations	What are the options for the opcode?
Type & size of operands	How is typing done? How is the size specified?

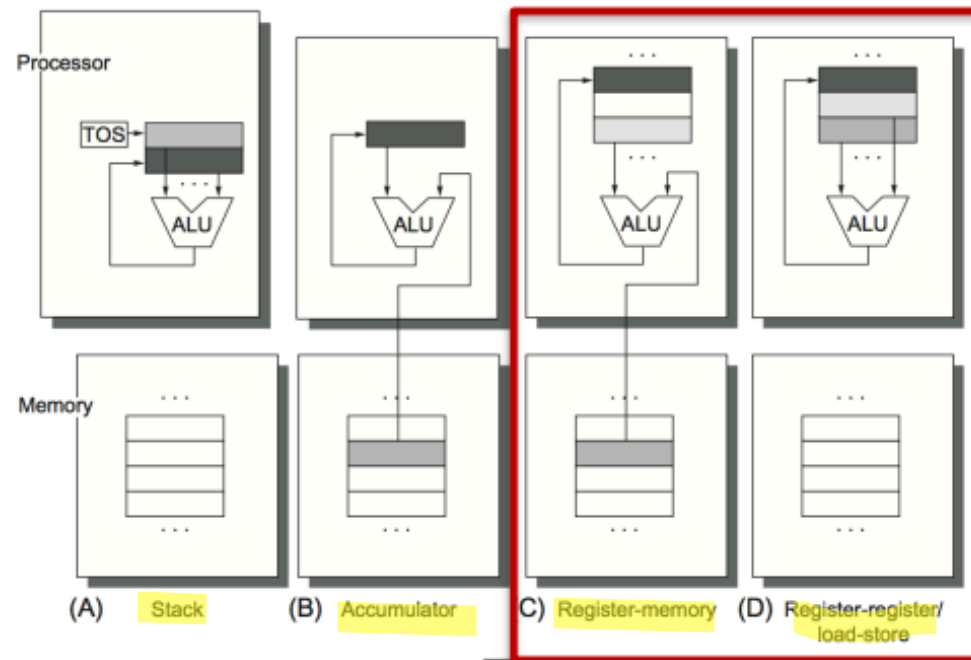
These choices critically affect number of instructions, CPI, and CPU cycle time

- Most basic differentiation: internal storage in a processor
 - Operands may be named **explicitly** or **implicitly**



- Major choices:
 1. In an **accumulator architecture** one operand is *implicitly* the accumulator => similar to calculator
 2. The operands in a **stack architecture** are *implicitly* on the top of the stack
 3. The **general-purpose register architectures** have only *explicit* operands – **either registers or memory location**

Four ISA Classes



Stack	Accumulator	Register (register-memory)	Register (load-store)
Push A	Load A	Load R1, A	Load R1, A
Push B	Add B	Add R3, R1, B	Load R2, B
Add	Store C	Store R3, C	Add R3, R1, R2
Pop C			Store R3, C

Figure A.2 The code sequence for $C = A + B$ for four classes of instruction sets. Note that the Add instruction has implicit operands for stack and accumulator architectures and explicit operands for register architectures. It is assumed that A, B, and C all belong in memory and that the values of A and B cannot be destroyed. Figure A.1 shows the Add operation for each class of architecture.

- How many registers are sufficient?
- General-purpose registers vs. special-purpose registers
 - *compiler flexibility and hand-optimization*
- Two major concerns for arithmetic and logical instructions (ALU)
 1. Two or three operands
 - $X + Y \Rightarrow X$
 - $X + Y \Rightarrow Z$
 2. How many of the operands may be memory addresses (0 – 3)

Number of memory addresses	Maximum number of operands allowed	Type of Architecture	Examples
0	3	Load-Store	Alpha, ARM, MIPS, PowerPC, SPARC, SuperH, TM32
1	2	Register-Memory	IBM 360/370, Intel 80x86, Motorola 68000, TI TMS320C54x
2	2	Memory – memory	VAX (also has 3 operand formats)
3	3	Memory - memory	VAX (also has 2 operand formats)

Hence, register architecture classification (# mem, # operands)₀

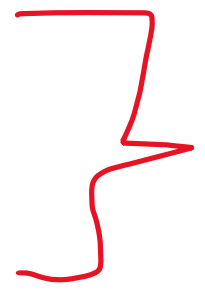
Advantages of Registers Rich CPU Architecture



More importantly, registers can be used to hold variables. When variables are allocated to registers, the memory traffic reduces, the program speeds up (because registers are faster than memory), and the code density improves (because a register can be named with fewer bits than can a memory location).

As explained in [Section A.8](#), compiler writers would prefer that all registers be equivalent and unreserved. Older computers compromise this desire by dedicating registers to special uses, effectively decreasing the number of general-purpose registers. If the number of truly general-purpose registers is too small, trying to allocate variables to registers will not be profitable. Instead, the compiler will reserve all the uncommitted registers for use in expression evaluation.

How many registers are sufficient? The answer, of course, depends on the effectiveness of the compiler. Most compilers reserve some registers for expression evaluation, use some for parameter passing, and allow the remainder to be allocated to hold variables. Modern compiler technology and its ability to effectively use larger numbers of registers has led to an increase in register counts in more recent architectures.



Tradeoffs for the ISA Classes

Type	Advantages	Disadvantages
Register-register (0,3)	Simple, fixed length instruction encoding. Simple code generation model. Instructions take similar numbers of clocks to execute.	Higher instruction count than architectures with memory references in the instructions. More instructions and lower instruction density leads to larger programs
Register-memory (1,2)	Data can be accessed without a separate load instruction first. Instruction format tends to be easy to encode and yields good density	Operands are not equivalent since a source operand is destroyed. Encoding a register number and a memory address in each instruction may restrict the number of registers. Clocks per instruction vary by operand location
Memory-memory (2,2) or (3,3)	Most compact. Does not waste registers for temporaries.	Large variation in instruction size, especially for three-operand instructions. In addition, large variation in work per instruction. Memory accesses create memory bottleneck. (Not used today)

(0,3) Register-Register RISC Architecture

- ALU is Register to Register – also known as
 - pure Reduced Instruction Set Computer (RISC)
- Advantages
 - Simple fixed length instruction encoding
 - Decode is simple since instruction types are small
 - Simple code generation model
 - Instruction CPI tends to be very uniform
 - Except for memory instructions of course
 - but there are only 2 of them - load and store
- Disadvantages
 - Instruction count tends to be higher
 - Some instructions are short - wasting instruction word bits

```

16 .L3:
17     lw     a5,-20(s0)
18     sll    a5,a5,2
19     ld     a4,-48(s0)
20     add    a5,a4,a5
21     flw    fa4,0(a5)
22     flw    fa5,-40(s0)
23     fmul.s  fa5,fa4,fa5
24     flw    fa4,-24(s0)
25     fadd.s  fa5,fa4,fa5
26     fsw     fa5,-24(s0)
27     lw     a5,-20(s0)
28     addw    a5,a5,1
29     sw     a5,-20(s0)
30 .L2:
31     lw     a4,-20(s0)
32     lw     a5,-36(s0)
33     blt     a4,a5,.L3
  
```

Memory Addressing

- Objects have **byte addresses**
 - the number of bytes counted from the beginning of memory

- **Object Length:**
 - bytes (8 bits), half words (16 bits),
 - words (32 bits), and double words (64 bits).
 - The type is implied in opcode, e.g.,
 - LDB – load byte
 - LDW – load word, etc

0000 0000 0000 0000	0000	
0000 0000 0000 0001	0001	
0000 0000 0000 0010	0002	
0000 0000 0000 0011	0003	
0000 0000 0000 0100	0004	
0000 0000 0000 0101	0005	
		...
0000 0000 0100 1001	0049	
0000 0000 0100 1010	004A	
0000 0000 0100 1011	004B	
		...
1111 1111 1111 1111	FFFF	
Binary Address	Hex	Memory Bytes

- **Byte Ordering**
 - **Little Endian:** puts the byte whose address is xx00 at the least significant position in the word. (7,6,5,4,3,2,1,0)
 - **Big Endian:** puts the byte whose address is xx00 at the most significant position in the word. (0,1,2,3,4,5,6,7)
 - Problem occurs when exchanging data among machines with different orderings

Interpreting Memory Addresses

- Alignment Issues
 - Accesses to objects larger than a byte must be aligned.
 - An access to an object of size s bytes at byte address A is aligned if $A \bmod s = 0$.
 - Misalignment causes hardware complications
 - since memory is typically aligned on a word or a double-word boundary
 - Misalignment typically results in an alignment fault that must be handled by the OS
- Hence
 - byte address is anything - never misaligned
 - half word - even addresses - low order address bit = 0 (XXXXXX0) else trap
 - word - low order 2 address bits = 0 (XXXXXX00) else trap
 - double word - low order 3 address bits = 0 (XXXXX000) else trap

Aligned / Misaligned Addresses

Value of 3 low-order bits of byte address									
Width of object	0	1	2	3	4	5	6	7	
1 byte (byte)	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned	
2 bytes (half word)	Aligned		Aligned		Aligned		Aligned		
2 bytes (half word)		Misaligned		Misaligned		Misaligned		Misaligned	
4 bytes (word)	Aligned				Aligned				
4 bytes (word)		Misaligned				Misaligned			
4 bytes (word)			Misaligned				Misaligned		
4 bytes (word)				Misaligned					Misaligned
8 bytes (double word)	Aligned								
8 bytes (double word)		Misaligned							
8 bytes (double word)			Misaligned						
8 bytes (double word)				Misaligned					
8 bytes (double word)					Misaligned				
8 bytes (double word)						Misaligned			
8 bytes (double word)							Misaligned		
8 bytes (double word)								Misaligned	

Figure A.5 Aligned and misaligned addresses of byte, half-word, word, and double-word objects for byte-addressed computers. For each misaligned example some objects require two memory accesses to complete. Every aligned object can always complete in one memory access, as long as the memory is as wide as the object. The figure shows the memory organized as 8 bytes wide. The byte offsets that label the columns specify the low-order 3 bits of the address.

Addressing Modes in RISC-V ISA

Addressing Modes for RISC-V Data Transfers

The only data addressing modes are immediate and displacement, both with 12-bit fields. Register indirect is accomplished simply by placing 0 in the 12-bit displacement field, and limited absolute addressing with a 12-bit field is accomplished by using register 0 as the base register. Embracing zero gives us four effective modes, although only two are supported in the architecture.

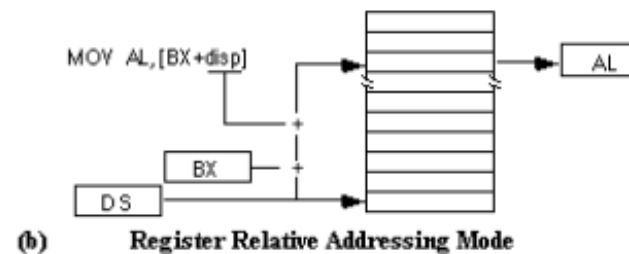
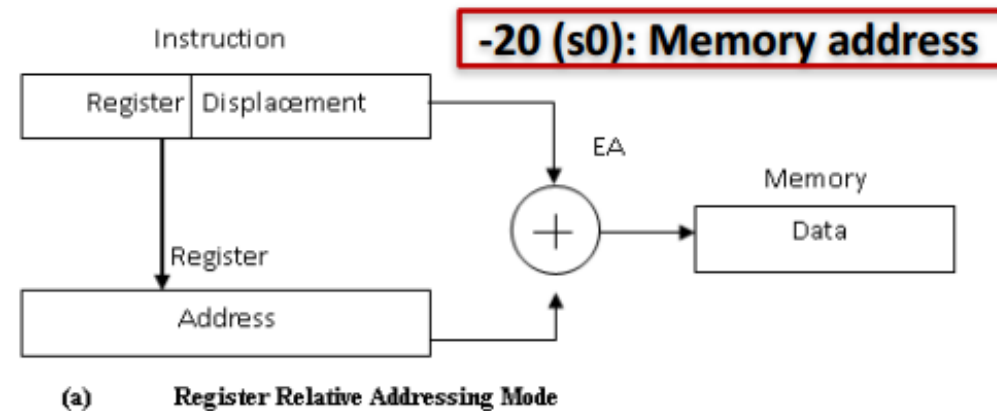
RV64G memory is byte addressable with a 64-bit address and uses Little Endian byte numbering. As it is a load-store architecture, all references between memory and either GPRs or FPRs are through loads or stores. Supporting the data types mentioned herein, memory accesses involving GPRs can be to a byte, half word, word, or double word. The FPRs may be loaded and stored with single-precision or double-precision numbers. Memory accesses need not be aligned; however, it may be that unaligned accesses run extremely slow. In practice, programmers and compilers would be stupid to use unaligned accesses.

Addressing Modes

- How architecture specify the **effective address** of an object?
 - Effective address: the actual memory address specified by the addressing mode.
 - E.g. $\text{Mem}[\text{R}[\text{R1}]]$ refers to the contents of the memory location whose location is given by the contents of register 1 (R1).

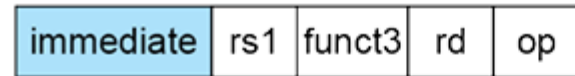
- Addressing Modes:**

- Register.
- Immediate
- Displacement
- Register indirect,.....

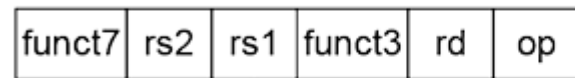


RISC-V Addressing Summary

1. Immediate addressing



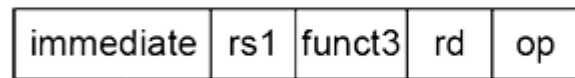
2. Register addressing



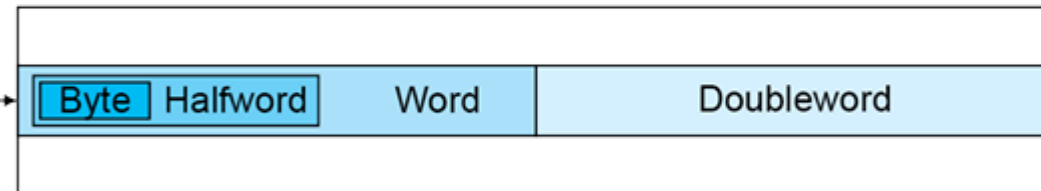
Registers

Register

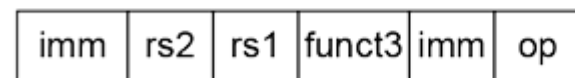
3. Base addressing, i.e., displacement addressing



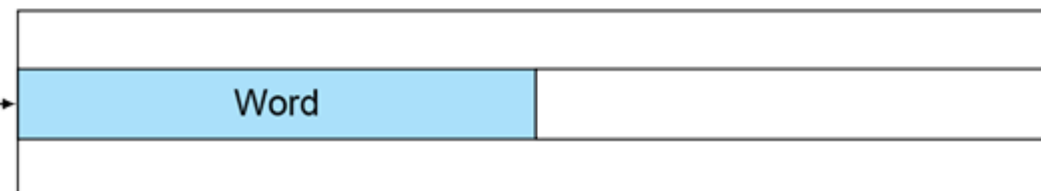
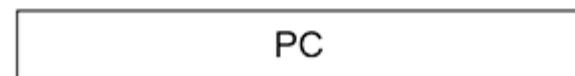
Memory



4. PC-relative addressing



Memory



Addressing Modes Comparison

Addressing mode	Example instruction	Meaning	When used
Register	Add R4,R3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Regs}[R3]$	When a value is in a register.
Immediate	Add R4,#3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + 3$	For constants.
Displacement	Add R4,100(R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[100 + \text{Regs}[R1]]$	Accessing local variables (+ simulates register indirect, direct addressing modes).
Register indirect	Add R4,(R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[\text{Regs}[R1]]$	Accessing using a pointer or a computed address.
Indexed	Add R3,(R1 + R2)	$\text{Regs}[R3] \leftarrow \text{Regs}[R3] + \text{Mem}[\text{Regs}[R1] + \text{Regs}[R2]]$	Sometimes useful in array addressing: R1 = base of array; R2 = index amount.
Direct or absolute	Add R1,(1001)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[1001]$	Sometimes useful for accessing static data; address constant may need to be large.
Memory indirect	Add R1,@(R3)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Mem}[\text{Regs}[R3]]]$	If R3 is the address of a pointer p , then mode yields $*p$.
Autoincrement	Add R1,(R2)+	$\begin{aligned} \text{Regs}[R1] &\leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]] \\ \text{Regs}[R2] &\leftarrow \text{Regs}[R2] + d \end{aligned}$	Useful for stepping through arrays within a loop. R2 points to start of array; each reference increments R2 by size of an element, d .
Autodecrement	Add R1,-(R2)	$\begin{aligned} \text{Regs}[R2] &\leftarrow \text{Regs}[R2] - d \\ \text{Regs}[R1] &\leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]] \end{aligned}$	Same use as autoincrement. Autodecrement/-increment can also act as push/pop to implement a stack.
Scaled	Add R1,100(R2)[R3]	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[100 + \text{Regs}[R2] + \text{Regs}[R3] * d]$	Used to index arrays. May be applied to any indexed addressing mode in some computers.

Figure A.6 Selection of addressing modes with examples, meaning, and usage. In autoincrement/-decrement and scaled addressing modes, the variable d designates the size of the data item being accessed (i.e., whether the instruction is accessing 1, 2, 4, or 8 bytes). These addressing modes are only useful when the elements being accessed are adjacent in memory. RISC computers use displacement addressing to simulate register indirect with 0 for the address and to simulate direct addressing using 0 in the base register. In our measurements, we use the first name shown for each mode. The extensions to C used as hardware descriptions are defined on page A-36.

Impact of Addressing Modes



- Instruction counts
- Architecture Complexity
- CPI

Summary of Usage of Addressing Modes

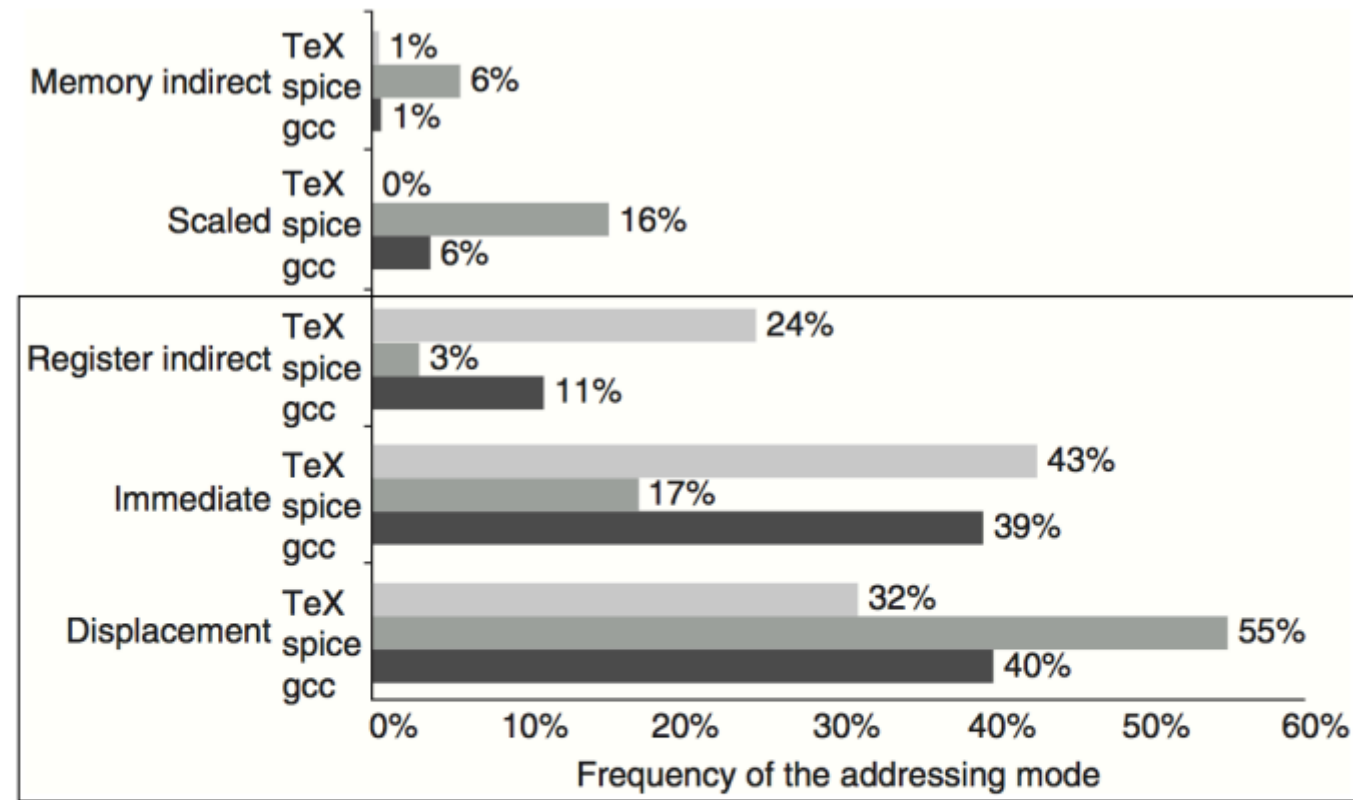


Figure A.7 Summary of use of memory addressing modes (including immediates). These major addressing modes account for all but a few percent (0% to 3%) of the memory accesses. Register modes, which are not counted, account for one-half of the operand references, while memory addressing modes (including immediate) account for the other half. Of course, the compiler affects what addressing modes are used; see [Section A.8](#). The memory indirect mode on the VAX can use displacement, autoincrement, or autodecrement to form the initial memory address; in these programs, almost

Displacement Addressing Mode



- **Benchmarks show**
 - 12 bits of displacement would capture about 75% of the full 32-bit displacements
 - 16 bits should capture about 99%
- **Remember:**
 - optimize for the common case. Hence, the choice is at least 12-16 bits
- For addresses that do fit in displacement size:
`Add R4, 10000 (R0)`
- For addresses that don't fit in displacement size, the compiler must do the following:
`Load R1, 1000000`
`Add R1, R0`
`Add R4, 0 (R1)`

- A new architecture expected to support at least:
displacement, immediate, and register indirect
 - represent 75% to 99% of the addressing modes
- The size of the address for displacement mode to be at least **12-16** bits
 - capture 75% to 99% of the displacements
- The size of the immediate field to be at least **8-16** bits
 - capture 50% to 80% of the immediates

Processors rely on compilers to generate codes using those addressing mode

Immediate Addressing Mode

- Used where we want to get to a numerical value in an instruction
- **Around 25% of the operations have an immediate operand**

At high level:

```
a = b + 3;
```

```
if ( a > 17 )
```

```
goto Addr
```

At Assembler level:

```
Load R2, #3
```

```
Add R0, R1, R2
```

```
Load R2, #17
```

```
CMPBGT R1, R2
```

```
Load R1, Address
```

```
Jump (R1)
```

4 Types and Sizes of Operands



How is the type of an operand designated?

- The type of the operand is usually encoded in the opcode
 - e.g., LDB – load byte; LDW – load word
- Common operand types: (imply their sizes)
 - Character (8 bits or 1 byte)
 - Half word (16 bits or 2 bytes)
 - Word (32 bits or 4 bytes)
 - Double word (64 bits or 8 bytes)
 - Single precision floating point (4 bytes or 1 word)
 - Double precision floating point (8 bytes or 2 words)
 - ✓ *Characters are almost always in ASCII*
 - ✓ *16-bit Unicode (used in Java) is gaining popularity*
 - ✓ *Integers are two's complement binary*
 - ✓ *Floating points follow the IEEE standard 754*
- Some architectures support *packed decimal*: 4 bits are used to encode the values 0-9; 2 decimal digits are packed into each byte

Distribution of Data Accesses vs Operand Size

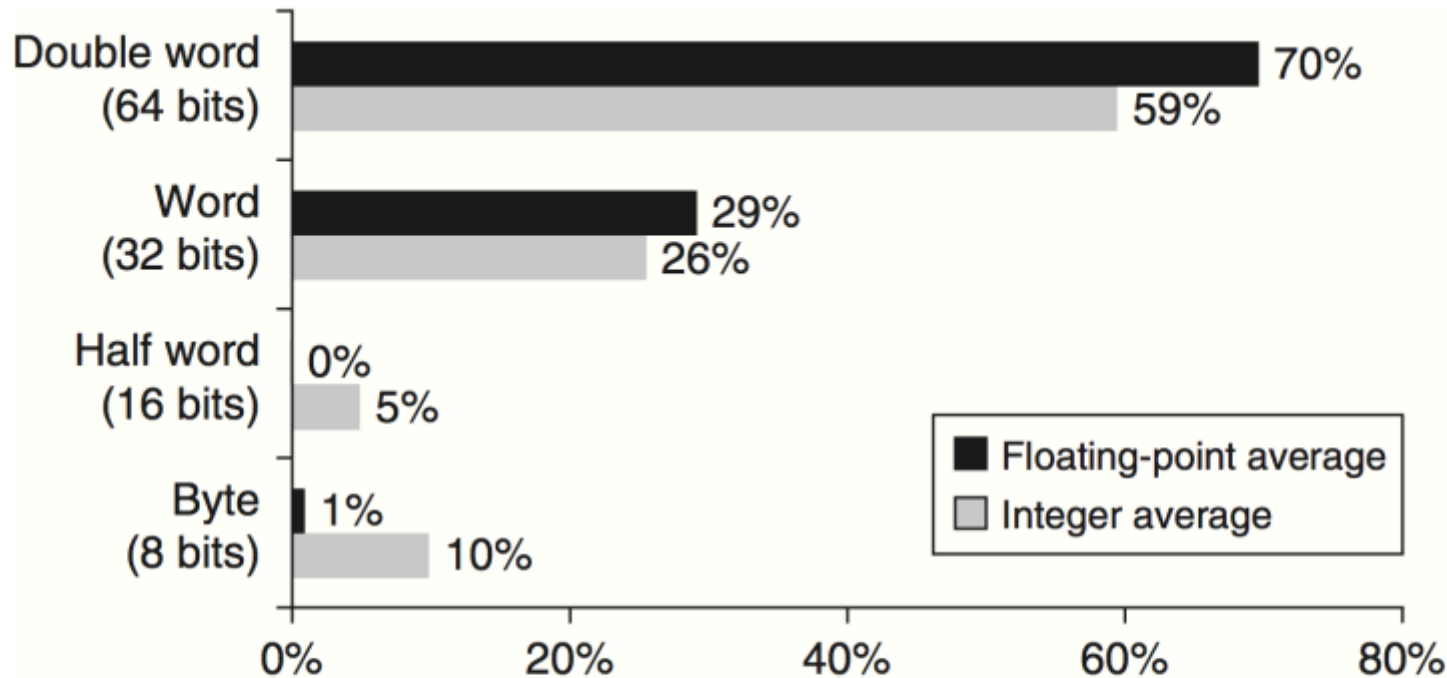


Figure A.11 Distribution of data accesses by size for the benchmark programs. The double-word data type is used for double-precision floating point in floating-point programs and for addresses, since the computer uses 64-bit addresses. On a 32-bit address computer the 64-bit addresses would be replaced by 32-bit addresses, and so almost all double-word accesses in integer programs would become single-word accesses.

Operations in the Instruction Set

Operator type	Examples
Arithmetic and logical	Integer arithmetic and logical operations: add, subtract, and, or, multiply, divide
Data transfer	Loads-stores (move instructions on computers with memory addressing)
Control	Branch, jump, procedure call and return, traps
System	Operating system call, virtual memory management instructions
Floating point	Floating-point operations: add, multiply, divide, compare
Decimal	Decimal add, decimal multiply, decimal-to-character conversions
String	String move, string compare, string search
Graphics	Pixel and vertex operations, compression/decompression operations

Figure A.12 Categories of instruction operators and examples of each. All computers generally provide a full set of operations for the first three categories. The support for system functions in the instruction set varies widely among architectures, but all computers must have some instruction support for basic system functions. The amount of support in the instruction set for the last four categories may vary from none to an extensive set of special instructions. Floating-point instructions will be provided in any

Top 10 Instructions for x86

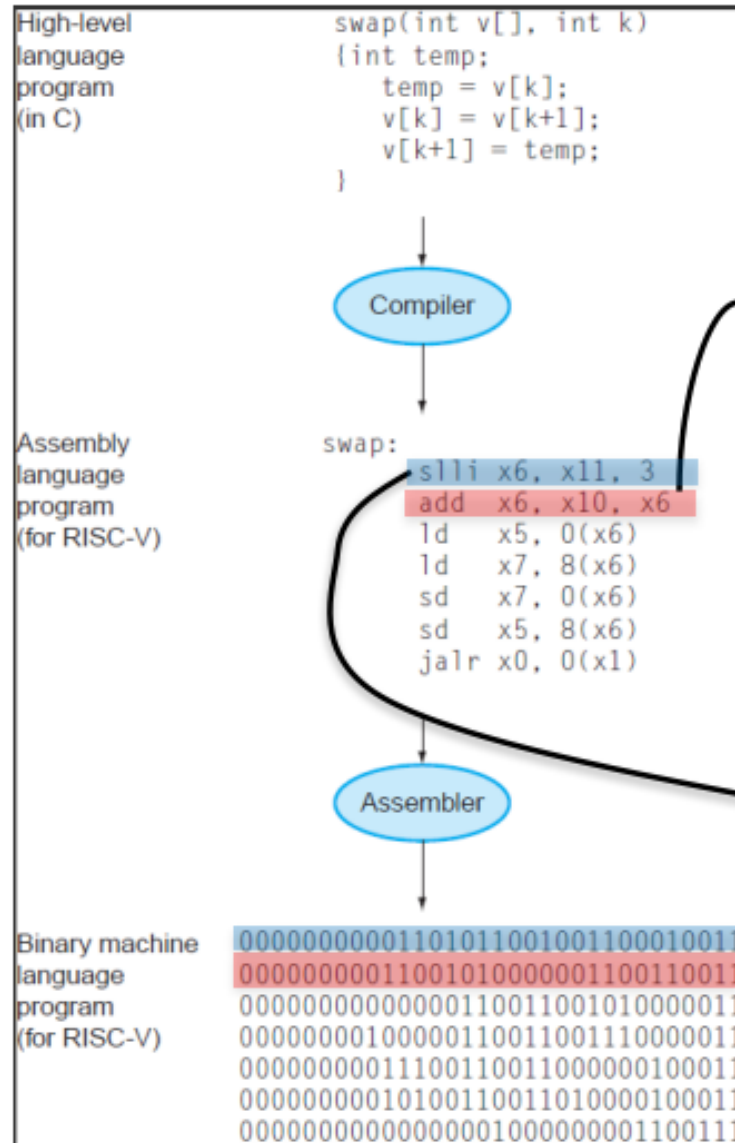


Rank	80x86 instruction	Integer average (% total executed)
1	load	22%
2	conditional branch	20%
3	compare	16%
4	store	12%
5	add	8%
6	and	6%
7	sub	5%
8	move register-register	4%
9	call	1%
10	return	1%
Total		96%

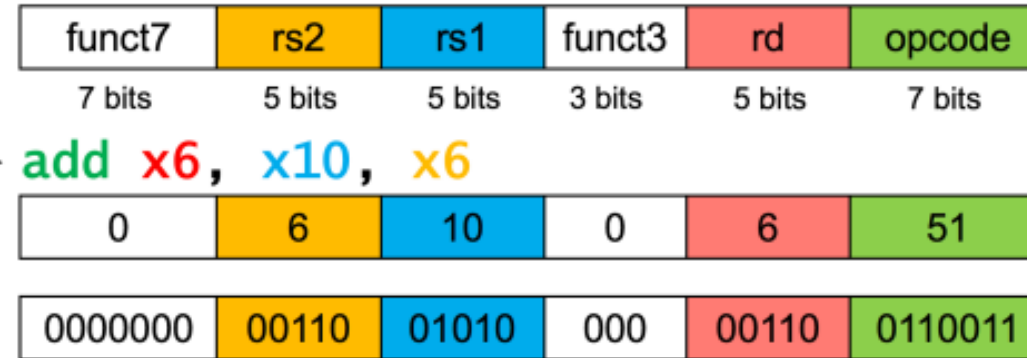
Figure A.13 The top 10 instructions for the 80x86. Simple instructions dominate this list and are responsible for 96% of the instructions executed. These percentages are the average of the five SPECint92 programs.

22

Instruction Encoding

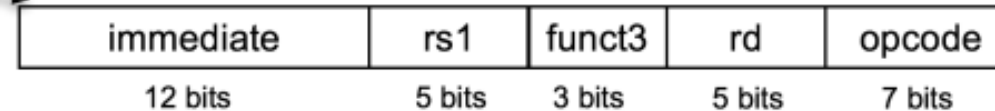


• RISC-V R-format instruction



0000 0000 0110 0101 0000 0011 0011 0011_{two} =
00650333₁₆

• RISC-V I-format instruction



Immediate arithmetic and load instructions

- rs1: source or base address register number
- immediate: constant operand, or offset added to base address
 - 2s-complement, sign extended

Control Flow Instructions

Instructions for Control Flow

- Control instructions change the flow of control:
 - instead of executing the next instruction, the program branches to the address specified in the branching instructions
- They break the pipeline
 - Difficult to optimize out
 - AND they are frequent
- Four types of control instructions
 - Conditional branches**
 - if...else, for/while, switch/case, ...
 - Jumps – unconditional transfer**
 - goto
 - Procedure calls**
 - foo()
 - Procedure returns**
 - return

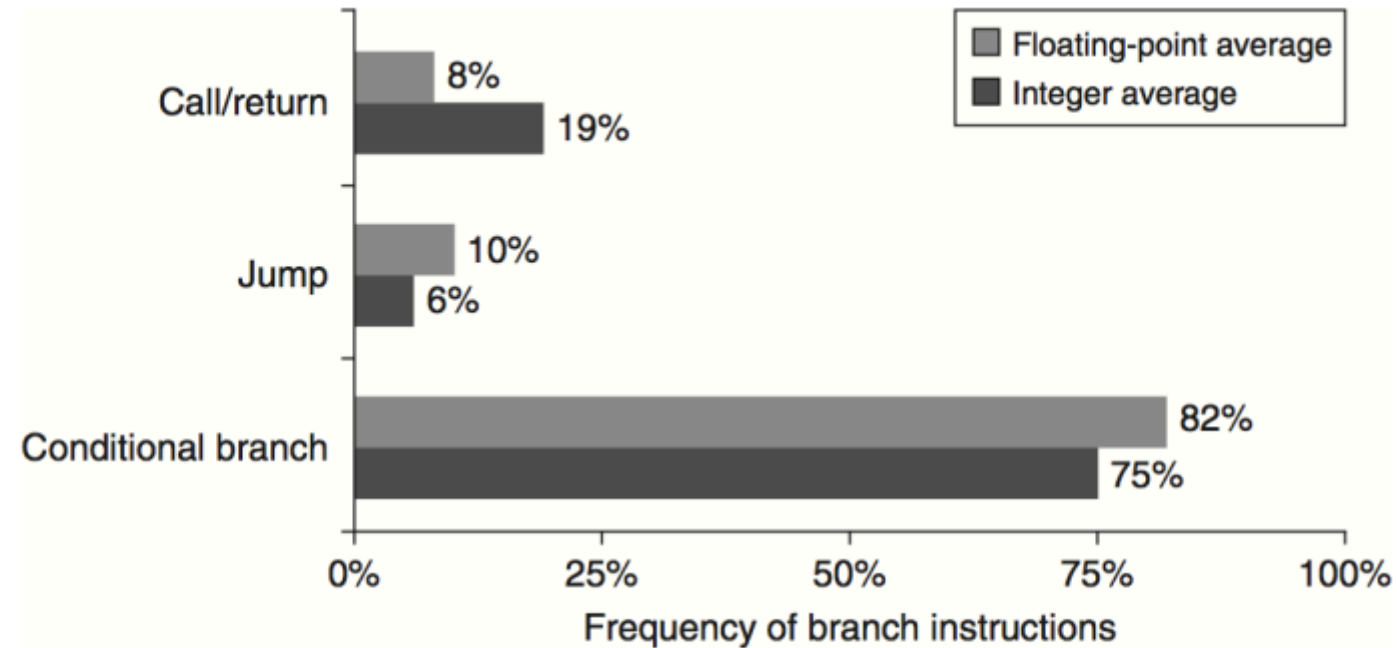
```

1 float sum(int N, float X[], float a) {
2     int i;
3     float result = 0.0;
4     for (i = 0; i < N; ++i)
5         result += a * X[i];
6     return result;
7 }
    
```

```

1      .file      "sum.c"
2      .text
3      .align    2
4      .globl   sum
5      .type     sum, @function
6  sum:
7      add      sp,sp,-48
8      sd      s0,40(sp)
9      add      s0,sp,48
10     sw      a0,-36(s0)
11     sd      a1,-48(s0)
12     fsw     fa2,-40(s0)
13     sw      zero,-24(s0)
14     sw      zero,-20(s0)
15     j       .L2
16  .L3:
17     lw      a5,-20(s0)
18     sll     a5,a5,2
19     ld      a4,-48(s0)
20     add     a5,a4,a5
21     flw     fa4,0(a5)
22     flw     fa5,-40(s0)
23     fmul.s  fa5,fa4,fa5
24     flw     fa4,-24(s0)
25     fadd.s  fa5,fa4,fa5
26     fsw     fa5,-24(s0)
27     lw      a5,-20(s0)
28     addw    a5,a5,1
29     sw      a5,-20(s0)
30  .L2:
31     lw      a4,-20(s0)
32     lw      a5,-36(s0)
33     blt     a4,a5,.L3
34     flw     fa5,-24(s0)
35     fmv.s   fa0,fa5
36     ld      s0,40(sp)
37     add     sp,sp,48
38     jr      ra
39     .size    sum,.-sum
40     .ident   "GCC: (GNU) 6.1.0"
    
```

Breakdown of Control Flow Instructions



- **Issues:**
 - Where is the target address? How to specify it? (label)
 - Caller: Where is return address kept? How are the arguments passed?
 - Callee: Where is return address? How are the results passed?

Addressing Modes for Control Instructions



- PC-relative (Program Counter)
 - Supply a displacement added to the PC
 - Known at compile time for jumps, branches, and calls (specified within the instruction)
 - The target is often near the current instruction
 - Requiring fewer bits
 - Independently of where it is loaded (position independence)
- Register indirect addressing – dynamic addressing
 - The target address may not be known at compile time
 - Naming a register that contains the target address
 - Case or switch statements
 - Virtual functions or methods in C++ or Java
 - High-order functions or function pointers in C or C++
 - Dynamically shared libraries

Conditional Branch Options

Name	Examples	How condition is tested	Advantages	Disadvantages
Condition code (CC)	80x86, ARM, PowerPC, SPARC, SuperH	Tests special bits set by ALU operations, possibly under program control.	Sometimes condition is set for free.	CC is extra state. Condition codes constrain the ordering of instructions since they pass information from one instruction to a branch.
Condition register	Alpha, MIPS	Tests arbitrary register with the result of a comparison.	Simple.	Uses up a register.
Compare and branch	PA-RISC, VAX	Compare is part of the branch. Often compare is limited to subset.	One instruction rather than two for a branch.	May be too much work per instruction for pipelined execution.

Figure A.16 The major methods for evaluating branch conditions, their advantages, and their disadvantages. Although condition codes can be set by ALU operations that are needed for other purposes, measurements on programs show that this rarely happens. The major implementation problems with condition codes arise when the condition code is set by a large or haphazardly chosen subset of the instructions, rather than being controlled by a bit in the instruction. Computers with compare and branch often limit the set of compares and use a condition register for more complex compares. Often, different techniques are used for branches based on floating-point comparison versus those based on integer comparison. This dichotomy is reasonable since the number of branches that depend on floating-point comparisons is much smaller than the number depending on integer comparisons.

- Procedure calls and returns
 - control transfer
 - state saving; the return address must be saved

Newer architectures require the compiler to generate stores and loads for each register saved and restored
- Two basic conventions in use to save registers
 - **caller saving**: the calling procedure must save the registers that it wants preserved for access after the call
 - the called procedure need not worry about registers
 - **callee saving**: the called procedure must save the registers it wants to use
 - leaving the caller unrestrained

most real systems today use a combination of both

 - Application binary interface (ABI) that set down the basic rules as to which register be caller saved and which should be callee saved

Instruction Encoding and Role of Compilers

Encoding an Instruction Set

- Opcode: specifying the operation
- # of operand
 - addressing mode
 - address specifier: tells what addressing mode is used
 - Load-store computer
 - Only one memory operand
 - Only one or two addressing modes
- The architecture must balancing several competing forces when encoding the instruction set:
 - # of registers & Addressing modes
 - Size of registers & Addressing mode fields
 - Average instruction size & Average program size.
 - Easy to handle in pipeline implementation.

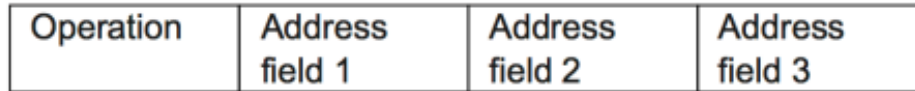


Three Basic Variations of Instruction Encoding



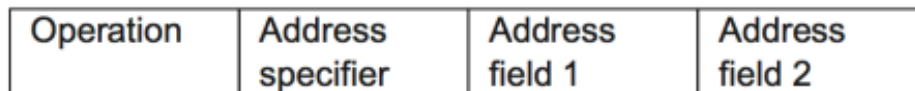
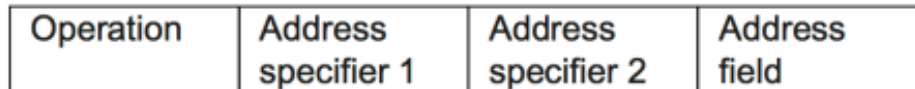
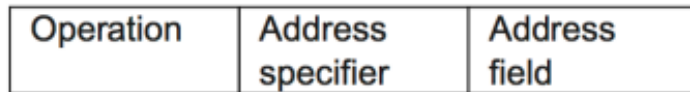
(A) Variable (e.g., Intel 80x86, VAX)

The length of 80x86 (CISC) instructions varies between 1 and 17 bytes.



(B) Fixed (e.g., RISC V, ARM, MIPS, PowerPC, SPARC)

The length of most RISC ISA instructions are 4 bytes.



(C) Hybrid (e.g., RISC V Compressed (RV32IC), IBM 360/370, microMIPS, Arm Thumb2)

X86 program are generally smaller than RISC ISA.

To reduce RISC code size

Figure A.18 Three basic variations in instruction encoding: variable length, fixed length, and hybrid. The variable format can support any number of operands, with each address specifier determining the addressing mode and the length of the specifier for that operand. It generally enables the smallest code representation, because

Instruction Length Tradeoff



- **Fixed length:** Length of all instructions the same
 - + Easier to decode single instruction in hardware
 - + Easier to decode multiple instructions concurrently
 - Wasted bits in instructions (Why is this bad?)
 - Harder-to-extend ISA (how to add new instructions?)
- **Variable length:** Length of instructions different (determined by opcode and sub-opcode)
 - + Compact encoding (Why is this good?)
 - Intel 432: Huffman encoding (sort of). 6 to 321 bit instructions. How?
 - More logic to decode a single instruction
 - Harder to decode multiple instructions concurrently
- **Tradeoffs**
 - Code size (memory space, bandwidth, latency) vs. hardware complexity
 - ISA extensibility and expressiveness
 - Performance? Smaller code vs. imperfect decode

Uniform Decoding vs Non-Uniform Decoding



- **Uniform decode:** Same bits in each instruction correspond to the same meaning
 - Opcode is always in the same location
 - immediate values, ...
 - Many “RISC” ISAs: Alpha, MIPS, SPARC
 - + Easier decode, simpler hardware
 - + Enables parallelism: generate target address before knowing the instruction is a branch
 - Restricts instruction format (fewer instructions?) or wastes space
- **Non-uniform decode**
 - E.g., opcode can be the 1st-7th byte in x86
 - + More compact and powerful instruction format
 - More complex decode logic

Reduced Code Size in RISC



- Hybrid encoding – support 16-bit and 32-bit instructions in RISC, eg. ARM Thumb, MIPS 16 and RISC-V
 - Narrow instructions support fewer operations, smaller address and immediate fields, fewer registers, and two-address format rather than the classic three-address format
 - Claim a code size reduction of up to 40%
- Compression in IBM's CodePack
 - Adds hardware to decompress instructions as they are fetched from memory on an instruction cache miss
 - The instruction cache contains full 32-bit instructions, but compressed code is kept in main memory, ROMs, and the disk
 - Claim code reduction 35% - 40%
 - PowerPC create a Hash table in memory that map between compressed and uncompressed address. Code size 35%~40%
- Hitachi's SuperH: fixed 16-bit format
 - 16 rather than 32 registers
 - fewer instructions

Different Choices of Instruction Encoding

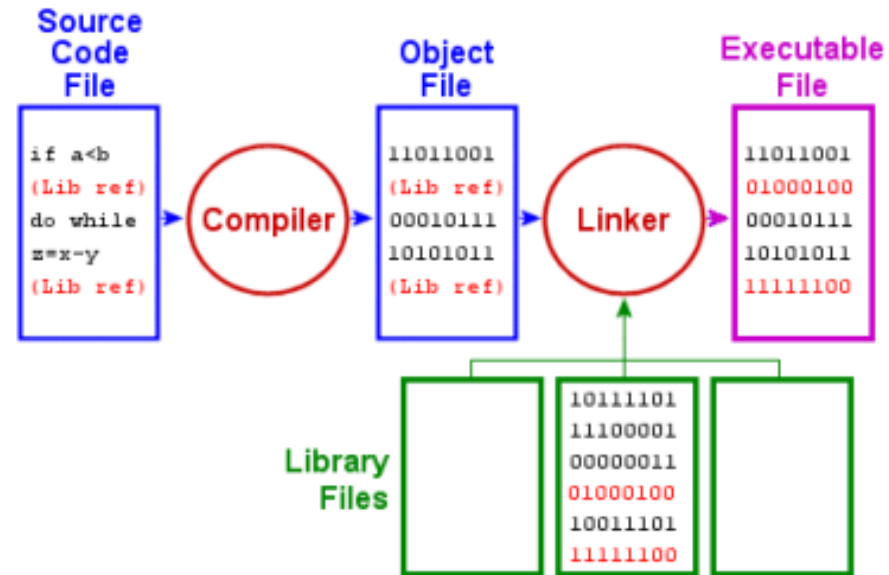


- Three choices
 - Variable, fixed and hybrid
 - Note the differences of hybrid and variable
- Choices of instruction encoding is a tradeoff between
 - For performance: fixed encoding
 - For code size: variable encoding
- How hybrid encoding is used in RISC to reduce code size
 - 16bit and 32bit
- In general, we see:
 - RISC: fixed or hybrid
 - CISC: variable

Role of Compilers

- Almost all programming is done in high-level languages.
 - An ISA is essentially a compiler target.

- *See backup slides for the compilation stage by most compiler, e.g. gcc*



- Compiler goals:
 - All correct programs execute correctly
 - Most compiled programs execute fast (optimizations)
 - Fast compilation
 - Debugging support

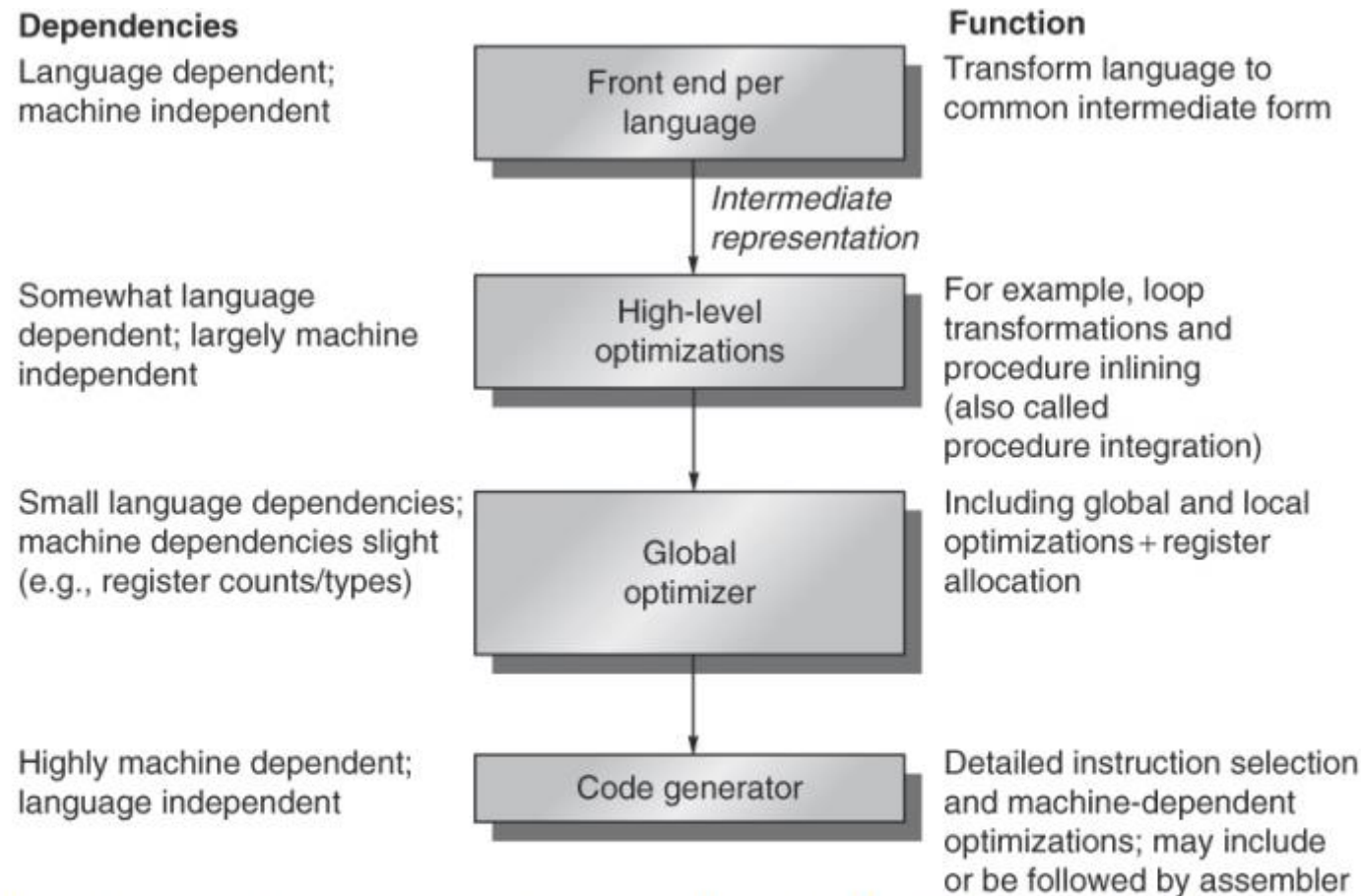


Figure A.19 Compilers typically consist of two to four passes, with more highly optimizing compilers having more passes. This structure maximizes the probability that a program compiled at various levels of optimization will produce the same output when given the same input. The optimizing passes are designed to be optional and may be skipped when faster compilation is the goal and lower-quality code is acceptable. A *pass* is simply one phase in which the compiler reads and transforms the entire program. (The term *phase* is often used inter-changeably with *pass*.) Because the optimizing passes are separated, multiple languages can use the same optimizing and code generation passes. Only a new front end is required for a new language.

5

Types of Compiler Optimizations



- High level – done at or near source code level
 - If procedure is called only once, put it in-line and save CALL
 - more general case: if call-count < some threshold, put them in-line
- Local – done within straight-line code
 - common sub-expressions produce same value – either allocate a register or replace with single copy
 - constant propagation – replace constant valued variable with the constant
 - stack height reduction – re-arrange expression tree to minimize temporary storage needs
- Global – across a branch
 - copy propagation – replace all instances of a variable A that has been assigned X (i.e., $A=X$) with X.
 - code motion – remove code from a loop that computes same value each iteration of the loop and put it before the loop
 - simplify or eliminate array addressing calculations in loops

Major Types of Compiler Optimizations

Optimization name	Explanation	Percentage of the total number of optimizing transforms
<i>High-level</i>	<i>At or near the source level; processor-independent</i>	
Procedure integration	Replace procedure call by procedure body	N.M.
<i>Local</i>	<i>Within straight-line code</i>	
Common subexpression elimination	Replace two instances of the same computation by single copy	18%
Constant propagation	Replace all instances of a variable that is assigned a constant with the constant	22%
Stack height reduction	Rearrange expression tree to minimize resources needed for expression evaluation	N.M.
<i>Global</i>	<i>Across a branch</i>	
Global common subexpression elimination	Same as local, but this version crosses branches	13%
Copy propagation	Replace all instances of a variable A that has been assigned X (i.e., $A = X$) with X	11%
Code motion	Remove code from a loop that computes same value each iteration of the loop	16%
Induction variable elimination	Simplify/eliminate array addressing calculations within loops	2%
<i>Processor-dependent</i>	<i>Depends on processor knowledge</i>	
Strength reduction	Many examples, such as replace multiply by a constant with adds and shifts	N.M.
Pipeline scheduling	Reorder instructions to improve pipeline performance	N.M.
Branch offset optimization	Choose the shortest branch displacement that reaches target	N.M.

Figure A.20 Major types of optimizations and examples in each class. These data tell us about the relative frequency of occurrence of various optimizations. The third column lists the static frequency with which some of the

Comparing Some Real ISA



Arch	Type	# Oper	# Mem	Data Size	# Regs	Addr Size	Use
Alpha	Reg-Reg	3	0	64-bit	32	64-bit	Workstation
ARM	Reg-Reg	3	0	32/64-bit	16	32/64-bit	Cell Phones, Embedded
MIPS	Reg-Reg	3	0	32/64-bit	32	32/64-bit	Workstation, Embedded
SPARC	Reg-Reg	3	0	32/64-bit	24-32	32/64-bit	Workstation
TI C6000	Reg-Reg	3	0	32-bit	32	32-bit	DSP
IBM 360	Reg-Mem	2	1	32-bit	16	24/31/64	Mainframe
x86	Reg-Mem	2	1	8/16/32/ 64-bit	4/8/24	16/32/64	Personal Computers
VAX	Mem-Mem	3	3	32-bit	16	32-bit	Minicomputer

Base Integer Instructions

Category	Name	Fmt	
Loads	Load Byte	I LB	rd,rs1,imm
	Load Halfword	I LH	rd,rs1,imm
	Load Word	I L{W D Q}	rd,rs1,imm
	Load Byte Unsigned	I LBU	rd,rs1,imm
	Load Half Unsigned	I L{H W D}U	rd,rs1,imm
Stores	Store Byte	S SB	rs1,rs2,imm
	Store Halfword	S SH	rs1,rs2,imm
	Store Word	S S{W D Q}	rs1,rs2,imm
Shifts	Shift Left	R SLL{W D}	rd,rs1,rs2
	Shift Left Immediate	I SLLI{W D}	rd,rs1,shamt
	Shift Right	R SRL{W D}	rd,rs1,rs2
	Shift Right Immediate	I SRLI{W D}	rd,rs1,shamt
	Shift Right Arithmetic	R SRA{W D}	rd,rs1,rs2
	Shift Right Arith Imm	I SRAI{W D}	rd,rs1,shamt
Arithmetic	ADD	R ADD{W D}	rd,rs1,rs2
	ADD Immediate	I ADDI{W D}	rd,rs1,imm
	SUBtract	R SUB{W D}	rd,rs1,rs2
	Load Upper Imm	U LUI	rd,imm
	Add Upper Imm to PC	U AUIPC	rd,imm
Logical	XOR	R XOR	rd,rs1,rs2
	XOR Immediate	I XORI	rd,rs1,imm
	OR	R OR	rd,rs1,rs2
	OR Immediate	I ORI	rd,rs1,imm
	AND	R AND	rd,rs1,rs2
	AND Immediate	I ANDI	rd,rs1,imm
Compare	Set <	R SLT	rd,rs1,rs2
	Set < Immediate	I SLTI	rd,rs1,imm
	Set < Unsigned	R SLTU	rd,rs1,rs2
	Set < Imm Unsigned	I SLTIU	rd,rs1,imm
Branches	Branch =	SB BEQ	rs1,rs2,imm
	Branch ≠	SB BNE	rs1,rs2,imm
	Branch <	SB BLT	rs1,rs2,imm
	Branch ≥	SB BGE	rs1,rs2,imm
	Branch < Unsigned	SB BLTU	rs1,rs2,imm
	Branch ≥ Unsigned	SB BGEU	rs1,rs2,imm
Jump & Link	J&L	UJ JAL	rd,imm
	Jump & Link Register	I JALR	rd,rs1,imm
Synch	Synch thread	I FENCE	
	Synch Instr & Data	I FENCE.I	
System	System CALL	I SCALL	
	System BREAK	I SBREAK	
Counters	Read CYCLE	I RDCYCLE	rd
	Read CYCLE upper Half	I RDCYCLEH	rd
	Read TIME	I RDTIME	rd
	Read TIME upper Half	I RDTIMEH	rd
	Read INSTR RETired	I RDINSTRET	rd
	Read INSTR upper Half	I RDINSTRETH	rd

	31	30	25	24	21	20	19	15	14	12	11	8	7	6	
R	funct7				rs2				rs1	funct3		rd		opcode	
I	imm[11:0]								rs1	funct3		rd		opcode	
S	imm[11:5]				rs2				rs1	funct3		imm[4:0]		opcode	
SB	imm[12]	imm[10:5]			rs2				rs1	funct3		imm[4:1]	imm[1]		opcode
U	imm[31:12]												rd		opcode
UJ	imm[20]	imm[10:1]			imm[11]		imm[19:12]						rd		opcode

Base Integer Instructions (32 64 128)					RV Privileged Instructions (32 64 128)					3 Optional FP Extensions: RV32{F D Q}					Optional Compressed Instructions: RVC													
Category	Name	Fmt	RV{32 64 128}I Base		Category	Name	Fmt	RV mnemonic		Category	Name	Fmt	RV{F D Q} (HP/SP,DP,QP)		Category	Name	Fmt	RVC										
Loads	Load Byte	I	LB	rd,rs1,imm	CSR Access	Atomic R/W	R	CSR{RW}	rd,csr,rs1	Load	Load	I	FL{W,D,Q}	rd,rs1,imm	Loads	Load Word	CL	C.LW	rd',rs1',imm									
	Load Halfword	I	LH	rd,rs1,imm		Atomic Read & Set Bit	R	CSR{RS}	rd,csr,rs1		Store	Store	S	FS{W,D,Q}		rs1,rs2,imm	Load Word SP	CI	C.LWSP	rd,imm								
	Load Word	I	L{W D Q}	rd,rs1,imm		Atomic Read & Clear Bit	R	CSR{RC}	rd,csr,rs1			Arithmetic	ADD	R	FADD.{S D Q}	rd,rs1,rs2	Load Double	CL	C.LD	rd',rs1',imm								
	Load Byte Unsigned	I	LBU	rd,rs1,imm		Atomic R/W Imm	R	CSR{RWI}	rd,csr,imm		SUBtract		R	FSUB.{S D Q}	rd,rs1,rs2	Load Double SP	CI	C.LWSP	rd,imm									
	Load Half Unsigned	I	L{H W D U}	rd,rs1,imm		Atomic Read & Set Bit Imm	R	CSR{RSI}	rd,csr,imm		MULTiply		R	FMUL.{S D Q}	rd,rs1,rs2	Load Quad	CL	C.LQ	rd',rs1',imm									
Stores	Store Byte	S	SB	rs1,rs2,imm	Atomic Read & Clear Bit Imm	R	CSR{RCI}	rd,csr,imm	DIVide	R	FDIV.{S D Q}	rd,rs1,rs2	Load Quad SP	CI	C.LQSP	rd,imm												
	Store Halfword	S	SH	rs1,rs2,imm	Change Level	Env. Call	R	ECALL	SQuare RooT	R	FSQRT.{S D Q}	rd,rs1	Load Byte Unsigned	CL	C.LBU	rd',rs1',imm												
	Store Word	S	S{W D Q}	rs1,rs2,imm		Environment Breakpoint	R	EBREAK	Negative Multiply-SUBtract	R	FMSUB.{S D Q}	rd,rs1,rs2,rs3	Float Load Word	CL	C.FLW	rd',rs1',imm												
Shifts	Shift Left	R	SLL{W D}	rd,rs1,rs2		Environment Return	R	ERET	Negative Multiply-SUBtract	R	FMSUB.{S D Q}	rd,rs1,rs2,rs3	Float Load Double	CL	C.FLD	rd',rs1',imm												
	Shift Left Immediate	I	SLLI{W D}	rd,rs1,shamt	Trap Redirect to Supervisor	MRTS	R	MRTS	Negative Multiply-ADD	R	FMNADD.{S D Q}	rd,rs1,rs2,rs3	Float Load Word SP	CI	C.FLWSP	rd,imm												
	Shift Right	R	SRL{W D}	rd,rs1,rs2		Redirect Trap to Hypervisor	R	M{RTH}	rd,rs1,rs2	Sign Inject	SIGN source	R	FSGNJ.{S D Q}	rd,rs1,rs2	Float Load Double SP	CI	C.FLDSP	rd,imm										
	Shift Right Immediate	I	SRLI{W D}	rd,rs1,shamt	Hypervisor Trap to Supervisor	R	HRTS	rd,rs1,rs2	Negative SiGN source		R	FSGNJN.{S D Q}	rd,rs1,rs2	Stores	Store Word	CS	C.SW	rs1',rs2',imm										
	Shift Right Arithmetic	R	SRA{W D}	rd,rs1,rs2	Interrupt	Wait for Interrupt	R	WFI	Xor SIGN source	R	FSGNJX.{S D Q}	rd,rs1,rs2	Store Word SP		CSS	C.SWSP	rs2,imm											
Shift Right Arith Imm	I	SRAI{W D}	rd,rs1,shamt	Supervisor FENCE		R	SPENCE.VM	rs1	Min/Max	MINimum	R	FMIN.{S D Q}	rd,rs1,rs2	Store Double	CS	C.SD	rs1',rs2',imm											
Arithmetic	ADD	R	ADD{W D}	rd,rs1,rs2	Optional Multiply-Divide Extension: RV32M					MAXimum	R	FMAX.{S D Q}	rd,rs1,rs2	Store Double SP	CSS	C.SDSP	rs2,imm											
	ADD Immediate	I	ADDI{W D}	rd,rs1,imm	Category	Name	Fmt	RV32M (Multi-Div)						Compare Float	R	FEQ.{S D Q}	rd,rs1,rs2	Store Quad	CS	C.SQ	rs1',rs2',imm							
	SUBtract	R	SUB{W D}	rd,rs1,rs2										MULTiply upper Half	R	MUL{W D}	rd,rs1,rs2	Compare Float <	R	FLT.{S D Q}	rd,rs1,rs2	Store Quad SP	CSS	C.SQSP	rs2,imm			
	Load Upper Imm	U	LUI	rd,imm										MULTiply upper Half	R	MULH	rd,rs1,rs2	Compare Float <=	R	FLT.{S D Q}	rd,rs1,rs2	Float Store Word	CSS	C.FSW	rd',rs1',imm			
	Add Upper Imm to PC	U	AUIPC	rd,imm										MULTiply Half Sign/Uns	R	MULHSU	rd,rs1,rs2	Compare Float ≤	R	FLE.{S D Q}	rd,rs1,rs2	Float Store Double	CSS	C.FSD	rd',rs1',imm			
Logical	XOR	R	XOR	rd,rs1,rs2										MULTiply Half Sign/Uns	R	MULHSU	rd,rs1,rs2	Categorize	Classify Typ	R	FCLASS.{S D Q}	rd,rs1	Float Store Word SP	CSS	C.FSWSP	rd,imm		
	XOR Immediate	I	XORI	rd,rs1,imm	Divide	DIVide	R	DIV{W D}	rd,rs1,rs2	Move	Move from Integer	R	FMV.S.X	rd,rs1	Float Store Double SP	CSS	C.FSDSP		rd,imm									
	OR	R	OR	rd,rs1,rs2		DIVide Unsigned	R	DIVU	rd,rs1,rs2		Move to Integer	R	FMV.X.S	rd,rs1	Arithmetic	ADD	CR		C.ADD	rd,rs1								
	OR Immediate	I	ORI	rd,rs1,imm	Remainder	REMAinder	R	REM{W D}	rd,rs1,rs2	Convert from Integer	R	FCVT.{S D Q}.W	rd,rs1	ADD Word		CR	C.ADDW		rd',rs2'									
	AND	R	AND	rd,rs1,rs2		REMAinder Unsigned	R	REMU{W D}	rd,rs1,rs2	Convert from Int Unsigned	R	FCVT.{S D Q}.WU	rd,rs1	ADD Immediate	CI	C.ADDI	rd,imm											
Compare	AND Immediate	I	ANDI	rd,rs1,imm	Optional Atomic Instruction Extension: RVA					Convert to Int	R	FCVT.W.{S D Q}	rd,rs1	ADD Word Imm	CI	C.ADDIW	rd,imm											
	Set <	R	SLT	rd,rs1,rs2	Category	Name	Fmt	RV{32 64 128}A (Atomic)						Convert to Int Unsigned	R	FCVT.WU.{S D Q}	rd,rs1	ADD SP Imm * 16	CI	C.ADDI16SP	x0,imm							
	Set < Immediate	I	SLTI	rd,rs1,imm										Load	Load Reserved	R	LR.{W D Q}	rd,rs1	Configuration	Read Stat	R	FRCSR	rd	ADD SP Imm * 4	CIW	C.ADDI4SPN	rd',imm	
	Set < Unsigned	R	SLTU	rd,rs1,rs2											Store	Store Conditiona	R	SC.{W D Q}		rd,rs1,rs2	Read Rounding Mode	R	FRRM	rd	Load Immediate	CI	C.LI	rd,imm
	Set < Imm Unsigned	I	SLTIU	rd,rs1,imm												Swap	SWAP	R		AMOSWAP.{W D Q}	rd,rs1,rs2	Read Flags	R	FRFLAGS	rd	Load Upper Imm	CI	C.LUI
Branches	Branch =	SB	BEQ	rs1,rs2,imm													Add	ADD		R	AMOADD.{W D Q}	rd,rs1,rs2	Swap Status Req	R	FSCSR	rd,rs1	MoVe	CR
	Branch ≠	SB	BNE	rs1,rs2,imm	Logical	XOR	R	AMOXOR.{W D Q}	rd,rs1,rs2	Swap Rounding Mode	R	FSRM	rd,rs1					SUB		CR	C.SUB	rd',rs2'						
	Branch <	SB	BLT	rs1,rs2,imm		AND	R	AMOAND.{W D Q}	rd,rs1,rs2	Swap Flags	R	FSFLAGS	rd,rs1	SUB Word				CR	C.SUBW	rd',rs2'								
	Branch ≥	SB	BGE	rs1,rs2,imm		OR	R	AMoor.{W D Q}	rd,rs1,rs2	Swap Rounding Mode Imm	I	FSRMI	rd,imm	Logical	XOR			CS	C.XOR	rd',rs2'								
	Branch < Unsigned	SB	BLTU	rs1,rs2,imm		Min/Max	MINimum	R	AMOMIN.{W D Q}	rd,rs1,rs2	Swan Flags Imm	I	FSFLAGSI		rd,imm	OR		CS	C.OR	rd',rs2'								
Branch ≥ Unsigned	SB	BGEU	rs1,rs2,imm	MAXimum			R	AMOMAX.{W D Q}	rd,rs1,rs2	3 Optional FP Extensions: RV{64 128}{F D Q}					AND	CS	C.AND	rd',rs2'										
Jump & Link	J&L	UJ	JAL	rd,imm	MINimum Unsigned		R	AMOMINU.{W D Q}	rd,rs1,rs2	Category	Name	Fmt	RV{F D Q} (HP/SP,DP,QP)		AND Immediate	CB	C.ANDI	rd',rs2'										
	Jump & Link Register	I	JALR	rd,rs1,imm	MAXimum Unsigned		R	AMOMAXU.{W D Q}	rd,rs1,rs2						Move	Move from Integer	R	FMV.{D Q}.X	rd,rs1	Shifts	Shift Left Imm	CI	C.SLLI	rd,imm				
Synch	Synch thread	I	FENCE	rd,rs1,imm	16-bit (RVC) and 32-bit Instruction Formats					Move to Integer	R	FMV.X.{D Q}	rd,rs1	Shift Right Immediate		CB	C.SRLI	rd',imm										
	Synch Instr & Data	I	FENCE.I	rd,rs1,imm						Convert	Convert from In	R	FCVT.{S D Q}.{L T}	rd,rs1	Convert from Int Unsigned	R	FCVT.{S D Q}.{L T}U	rd,rs1	Shift Right Arith Imm	CB	C.SRAI	rd',imm						
System	System CALL	I	SCALL	rd,rs1,imm	Convert to Int	R	FCVT.{L T}.{S D Q}	rd,rs1	Convert to Int Unsigned		R	FCVT.{L T}U.{S D Q}	rd,rs1	Branches		Branch=0	CB	C.BEQZ	rs1',imm									
	System BREAK	I	SBREAK	rd,rs1,imm		16-bit (RVC) and 32-bit Instruction Formats					3 Optional FP Extensions: RV{64 128}{F D Q}					Branch≠0	CB	C.BNEZ	rs1',imm									
Counters	ReaD CYCLE	I	RDCYCLE	rd	CI											10 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0				Move	Move from Integer	R	FCVT.{S D Q}.{L T}	rd,rs1	Jump	Jump	CI	C.J
	ReaD CYCLE upper Half	I	RDCYCLEH	rd		CSS	func4	rd,rs1	rs2	op	Shifts	Jump Register	CR	C.JR	rd,rs1													
	ReaD TIME	I	RDTIME	rd			CIW	func3	imm	rd,rs1		imm	op	Branches	J&L						CJ	C.JAL	imm					
	ReaD TIME upper Half	I	RDTIMEH	rd				CL	func3	imm		rd'	op		System						Env. BREAK	CI	C.EBREAK					
	ReaD INSTR RETired	I	RDINSTRET	rd					CS	func3		imm	rs1'								imm	rs2'	op	Jump & Link		J&L	CJ	C.JAL
ReaD INSTR upper Half	I	RDINSTRETH	rd	CB	func3					offset		rs1'	offset			op	Jump & Link	J&L	CJ	C.JAL	rs1							

**EXTRA SLIDES – May Ignore
Further slides**

Operands in ISA

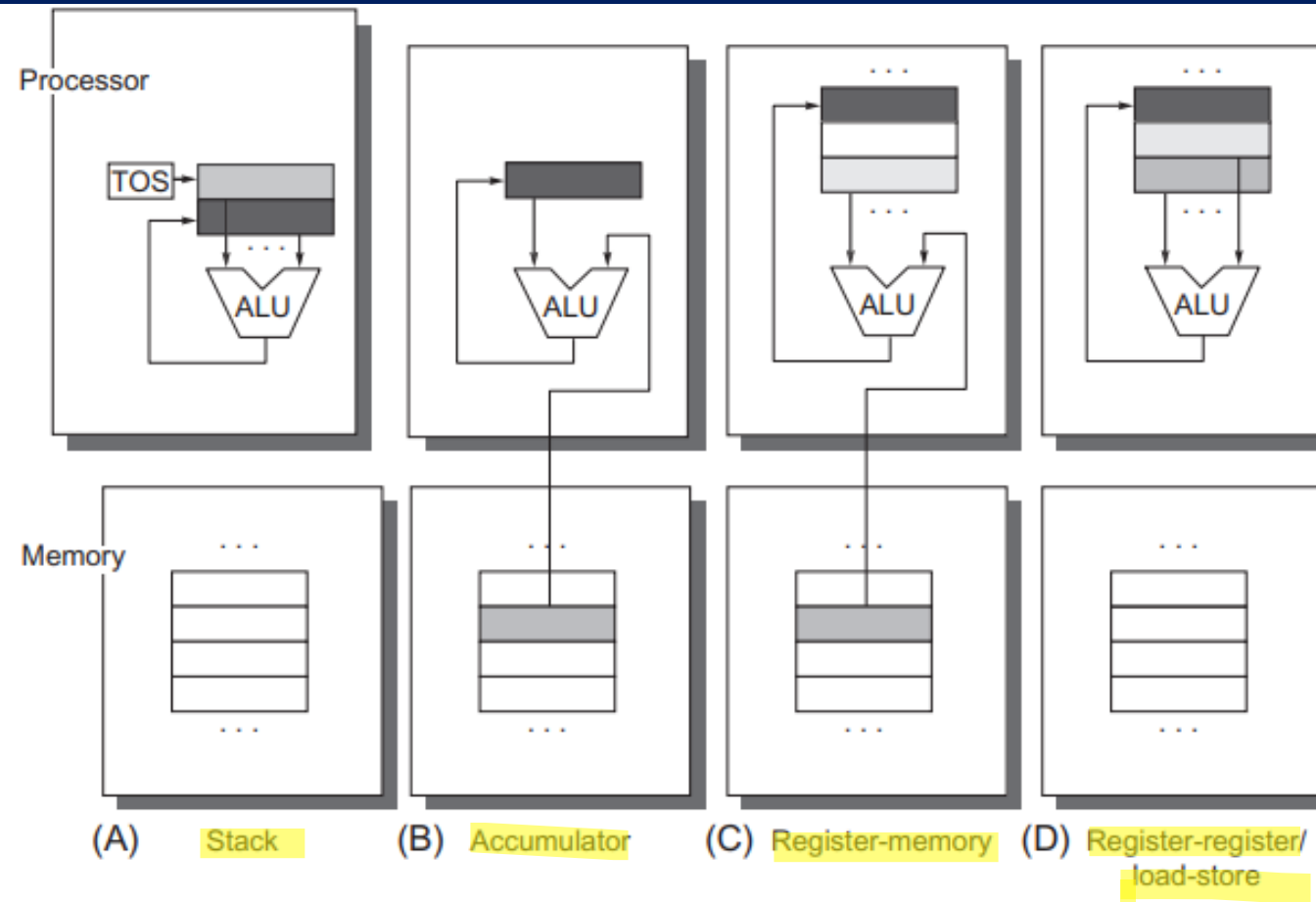


Figure A.1 Operand locations for four instruction set architecture classes. The arrows indicate whether the operand is an input or the result of the arithmetic-logical unit (ALU) operation, or both an input and result. Lighter shades indicate inputs, and the dark shade indicates the result. In (A), a top of stack (TOS) register points to the top input operand, which is combined with the operand below. The first operand is removed from the stack, the result takes the place of the second operand, and TOS is updated to point to the result. All operands are implicit. In (B), the accumulator is both an implicit input operand and a result. In (C), one input operand is a register, one is in memory, and the result goes to a register. All operands are registers in (D) and, like the stack architecture, can be transferred to memory only via separate instructions. Computer Architecture, 4th Edition, Fall 2006

Add Instruction in different Operand architecture



Stack	Accumulator	Register (register-memory)	Register (load-store)
Push A	Load A	Load R1 ,A	Load R1 ,A
Push B	Add B	Add R3 ,R1 ,B	Load R2 ,B
Add	Store C	Store R3 ,C	Add R3 ,R1 ,R2
Pop C			Store R3 ,C

Figure A.2 The code sequence for $C = A + B$ for four classes of instruction sets. Note that the Add instruction has implicit operands for stack and accumulator architectures and explicit operands for register architectures. It is assumed that A, B, and C all belong in memory and that the values of A and B cannot be destroyed. Figure A.1 shows the Add operation for each class of architecture.

Compare - Load Store Architecture



Number of memory addresses	Maximum number of operands allowed	Type of architecture	Examples
0	3	Load-store	ARM, MIPS, PowerPC, SPARC, RISC-V
1	2	Register-memory	IBM 360/370, Intel 80x86, Motorola 68000, TI TMS320C54x
2	2	Memory-memory	VAX (also has three-operand formats)
3	3	Memory-memory	VAX (also has two-operand formats)

Figure A.3 Typical combinations of memory operands and total operands per typical ALU instruction with examples of computers. Computers with no memory reference per ALU instruction are called load-store or register-register computers. Instructions with multiple memory operands per typical ALU instruction are called register-memory or memory-memory, according to whether they have one or more than one memory operand.

General Purpose Registers

Type	Advantages	Disadvantages
Register-register (0, 3)	Simple, fixed-length instruction encoding. Simple code generation model. Instructions take similar numbers of clocks to execute (see Appendix C)	Higher instruction count than architectures with memory references in instructions. More instructions and lower instruction density lead to larger programs, which may have some instruction cache effects
Register-memory (1, 2)	Data can be accessed without a separate load instruction first. Instruction format tends to be easy to encode and yields good density	Operands are not equivalent because a source operand in a binary operation is destroyed. Encoding a register number and a memory address in each instruction may restrict the number of registers. Clocks per instruction vary by operand location
Memory-memory (2, 2) or (3, 3)	Most compact. Doesn't waste registers for temporaries	Large variation in instruction size, especially for three-operand instructions. In addition, large variation in work per instruction. Memory accesses create memory bottleneck. (Not used today.)

Figure A.4 Advantages and disadvantages of the three most common types of general-purpose register computers. The notation (m, n) means m memory operands and n total operands. In general, computers with fewer alternatives simplify the compiler's task because there are fewer decisions for the compiler to make (see [Section A.8](#)). Computers with a wide variety of flexible instruction formats reduce the number of bits required to encode the program. The number of registers also affects the instruction size because you need $\log_2$ (number of registers) for each register specifier in an instruction. Thus, doubling the number of registers takes three extra bits for a register-register architecture, or about 10% of a 32-bit instruction.

Interpreting Memory Addresses

How is a memory address interpreted? That is, what object is accessed as a function of the address and the length? All the instruction sets discussed in this book are byte addressed and provide access for bytes (8 bits), half words (16 bits), and words (32 bits). Most of the computers also provide access for double words (64 bits).

There are two different conventions for ordering the bytes within a larger object. Little Endian byte order puts the byte whose address is “x ... x000” at the least-significant position in the double word (the little end). The bytes are numbered:



Big Endian byte order puts the byte whose address is “x ... x000” at the most-significant position in the double word (the big end). The bytes are numbered:



Addressing Modes

Addressing mode	Example instruction	Meaning	When used
Register	Add R4 , R3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Regs}[R3]$	When a value is in a register
Immediate	Add R4 , 3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + 3$	For constants
Displacement	Add R4 , 100(R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[100 + \text{Regs}[R1]]$	Accessing local variables (+ simulates register indirect, direct addressing modes)
Register indirect	Add R4 , (R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[\text{Regs}[R1]]$	Accessing using a pointer or a computed address
Indexed	Add R3 , (R1 + R2)	$\text{Regs}[R3] \leftarrow \text{Regs}[R3] + \text{Mem}[\text{Regs}[R1] + \text{Regs}[R2]]$	Sometimes useful in array addressing: R1 = base of array; R2 = index amount
Direct or absolute	Add R1 , (1001)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[1001]$	Sometimes useful for accessing static data; address constant may need to be large
Memory indirect	Add R1 , @(R3)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Mem}[\text{Regs}[R3]]]$	If R3 is the address of a pointer p , then mode yields $*p$
Autoincrement	Add R1 , (R2) +	$\begin{aligned} \text{Regs}[R1] &\leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]] \\ \text{Regs}[R2] &\leftarrow \text{Regs}[R2] + d \end{aligned}$	Useful for stepping through arrays within a loop. R2 points to start of array; each reference increments R2 by size of an element, d
Autodecrement	Add R1 , -(R2)	$\begin{aligned} \text{Regs}[R2] &\leftarrow \text{Regs}[R2] - d \\ \text{Regs}[R1] &\leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]] \end{aligned}$	Same use as autoincrement. Autodecrement/-increment can also act as push/pop to implement a stack.
Scaled	Add R1 , 100(R2)[R3]	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[100 + \text{Regs}[R2] + \text{Regs}[R3] * d]$	Used to index arrays. May be applied to any indexed addressing mode in some computers

Figure A.6 Selection of addressing modes with examples, meaning, and usage. In autoincrement/-decrement and scaled addressing modes, the variable d designates the size of the data item being accessed (i.e., whether the instruction is accessing 1, 2, 4, or 8 bytes). These addressing modes are only useful when the elements being accessed are adjacent in memory. RISC computers use displacement addressing to simulate register indirect with 0 for the address and to simulate direct addressing using 0 in the base register. In our measurements, we use the first name shown for each mode. The extensions to C used as hardware descriptions are defined on page A.38.

Summary: Memory Addressing

First, because of their popularity, we would expect a new architecture to support at least the following addressing modes: displacement, immediate, and register indirect. [Figure A.7](#) shows that they represent 75%–99% of the addressing modes used

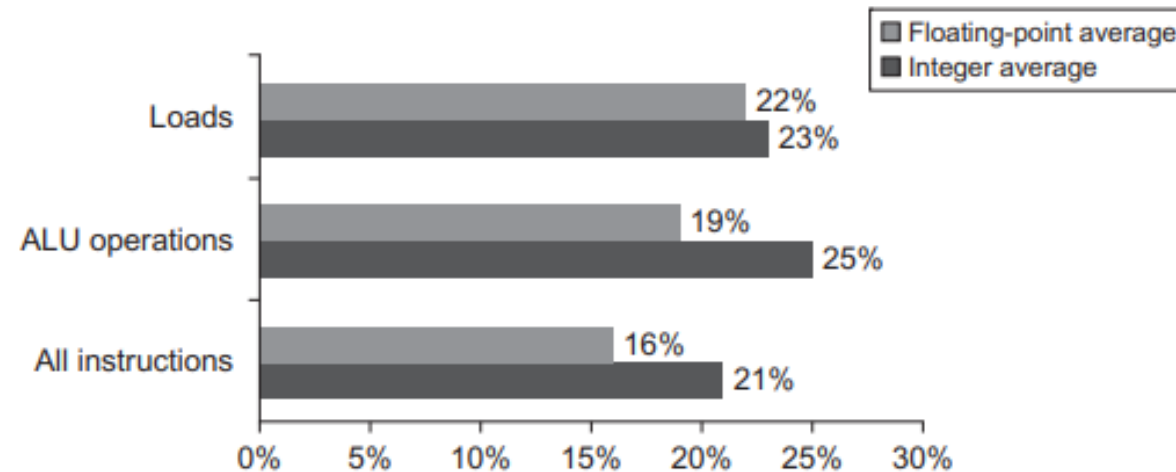


Figure A.9 About one-quarter of data transfers and ALU operations have an immediate operand. The bottom bars show that integer programs use immediates in about one-fifth of the instructions, while floating-point programs use immediates in about one-sixth of the instructions. For loads, the load immediate instruction loads 16 bits into either half of a 32-bit register. Load immediates are not loads in a strict sense because they do not access memory. Occasionally a pair of load immediates is used to load a 32-bit constant, but this is rare. (For ALU operations, shifts by a constant amount are included as operations with immediate operands.) The programs and computer used to collect these statistics are the same as in [Figure A.8](#).

Categories of Instruction Operators



Operator type	Examples
Arithmetic and logical	Integer arithmetic and logical operations: add, subtract, and, or, multiply, divide
Data transfer	Loads-stores (move instructions on computers with memory addressing)
Control	Branch, jump, procedure call and return, traps
System	Operating system call, virtual memory management instructions
Floating point	Floating-point operations: add, multiply, divide, compare
Decimal	Decimal add, decimal multiply, decimal-to-character conversions
String	String move, string compare, string search
Graphics	Pixel and vertex operations, compression/decompression operations

Figure A.12 Categories of instruction operators and examples of each. All computers generally provide a full set of operations for the first three categories. The support for system functions in the instruction set varies widely among architectures, but all computers must have some instruction support for basic system functions. The amount of support in the instruction set for the last four categories may vary from none to an extensive set of special instructions. Floating-point instructions will be provided in any computer that is intended for use in an application that makes much use of floating point. These instructions are sometimes part of an optional instruction set. Decimal and string instructions are sometimes primitives, as in the VAX or the IBM 360, or may be synthesized by the compiler from simpler instructions. Graphics instructions typically operate on many smaller data items in parallel—for example, performing eight 8-bit additions on two 64-bit operands.

Using SPEC Int Benchmark



Rank	80x86 instruction	Integer average % total executed)
1	Load	22%
2	Conditional branch	20%
3	Compare	16%
4	Store	12%
5	Add	8%
6	And	6%
7	Sub	5%
8	Move register-register	4%
9	Call	1%
10	Return	1%
Total		96%

Figure A.13 The top 10 instructions for the 80x86. Simple instructions dominate this list and are responsible for 96% of the instructions executed. These percentages are the average of the five SPECint92 programs.

Benchmark on Control Instructions

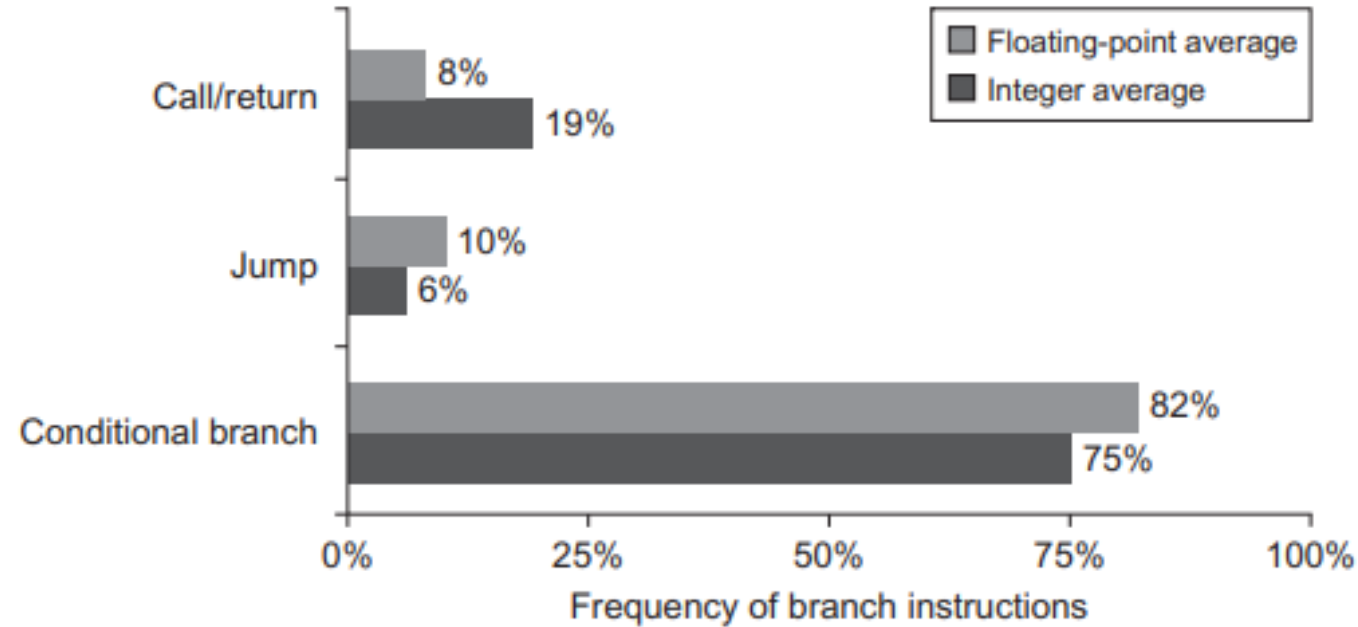


Figure A.14 Breakdown of control flow instructions into three classes: calls or returns, jumps, and conditional branches. Conditional branches clearly dominate. Each type is counted in one of three bars. The programs and computer used to collect these statistics are the same as those in [Figure A.8](#).

One of the most noticeable properties of branches is that a large number of the comparisons are simple tests, and a large number are comparisons with zero. Thus, some architectures choose to treat these comparisons as special cases, especially if a *compare and branch* instruction is being used. [Figure A.17](#) shows the frequency of different comparisons used for conditional branching.

Procedures and Functions

Procedure Invocation Options

```
int square(int x)
{
    int square_of_x;
    square_of_x = x * x;
    return square_of_x;
}

int square_of_10;
square_of_10 = square(10);
```

Procedure calls and returns include control transfer and possibly some state saving; at a minimum the **return address must be saved somewhere**, sometimes in a special link register or just a GPR. Some **older architectures provide a mechanism to save many registers**, while **newer architectures require the compiler to generate stores and loads for each register saved and restored**.

There are two basic conventions in use to save registers: either at the call site or inside the procedure being called. **Caller saving** means that the calling procedure must save the registers that it wants preserved for access after the call, and thus the called procedure need not worry about registers. **Callee saving** is the opposite: the called procedure must save the registers it wants to use, leaving the caller unrestrained. There are times when caller save must be used because of access patterns to globally visible variables in two different procedures. For example, suppose we have a procedure P1 that calls procedure P2, and both procedures manipulate the global variable *x*. If P1 had allocated *x* to a register, it must be sure to save *x* to a location known by P2 before the call to P2. A compiler's ability to discover when a called procedure may access register-allocated quantities is complicated by the possibility of separate compilation. Suppose P2 may not touch *x* but can call another procedure, P3, that may access *x*, yet P2 and P3 are compiled separately. Because of these complications, most compilers will conservatively caller save *any* variable that may be accessed during a call.

In the cases where either convention could be used, some programs will be more optimal with callee save and some will be more optimal with caller save. As a result, **most real systems today use a combination of the two mechanisms**. This convention is specified in an application binary interface (ABI) that sets down the basic rules as to which registers should be caller saved and which should be callee saved. Later in this appendix we will examine the mismatch between sophisticated instructions for automatically saving registers and the needs of the compiler.

**PUSH and
POP Instructions**

Suggestions for Architecture



We have now completed our instruction architecture tour at the level seen by an assembly language programmer or compiler writer. We are leaning toward a load-store architecture with displacement, immediate, and register indirect addressing modes. These data are 8-, 16-, 32-, and 64-bit integers and 32- and 64-bit floating-point data. The instructions include simple operations, PC-relative conditional branches, jump and link instructions for procedure call, and register indirect jumps for procedure return (plus a few other uses).

Now we need to select how to represent this architecture in a form that makes it easy for the hardware to execute.

Instruction Encoding Variety

Operation and no. of operands	Address specifier 1	Address field 1	...	Address specifier n	Address field n
----------------------------------	------------------------	--------------------	-----	--------------------------	----------------------

(A) Variable (e.g., Intel 80x86, VAX)

Operation	Address field 1	Address field 2	Address field 3
-----------	--------------------	--------------------	--------------------

(B) Fixed (e.g., RISC V, ARM, MIPS, PowerPC, SPARC)

Operation	Address specifier	Address field
-----------	----------------------	------------------

Operation	Address specifier 1	Address specifier 2	Address field
-----------	------------------------	------------------------	------------------

Operation	Address specifier	Address field 1	Address field 2
-----------	----------------------	--------------------	--------------------

(C) Hybrid (e.g., RISC V Compressed (RV32IC), IBM 360/370, microMIPS, Arm Thumb2)

Figure A.18 Three basic variations in instruction encoding: variable length, fixed length, and hybrid. The variable format can support any number of operands, with each address specifier determining the addressing mode and the length of the specifier for that operand. It generally enables the smallest code representation, because unused fields need not be included. The fixed format always has the same number of operands, with the addressing modes (if options exist) specified as part of the opcode. It generally results in the largest code size. Although the fields tend not to vary in their location, they will be used for different purposes by different instructions. The hybrid approach has multiple formats specified by the opcode, adding one or two fields to specify the addressing mode and one or two fields to specify the operand address.

Reduced Code Size in RISCs

As RISC computers started being used in embedded applications, the 32-bit fixed format became a liability because **use cost, and hence smaller code, are important**. In response, several manufacturers offered a new hybrid version of their RISC instruction sets, with both 16-bit and 32-bit instructions. The narrow instructions support fewer operations, smaller address and immediate fields, fewer registers, and the two-address format rather than the classic three-address format of RISC computers. RISC-V offers such an extension, called RV32IC, the C standing for compressed. Common instruction occurrences, such as intermediates with small values and common ALU operations with the source and destination register being identical, are encoded in 16-bit formats. Appendix K gives two other examples, the ARM Thumb and microMIPS, which both claim a code size reduction of up to 40%.

In contrast to these instruction set extensions, IBM simply compresses its standard instruction set and then adds hardware to decompress instructions as they are fetched from memory on an instruction cache miss. Thus, the instruction cache contains full 32-bit instructions, but compressed code is kept in main memory, ROMs, and the disk. The advantage of a compressed format, such as RV32IC, microMIPS and Thumb2 is that instruction caches act as if they are about 25% larger, while IBM's CodePack means that compilers need not be changed to handle different instruction sets and instruction decoding can remain simple.

- *High-level optimizations* are often done on the source with output fed to later optimization passes.
- *Local optimizations* optimize code only within a straight-line code fragment (called a *basic block* by compiler people).
- *Global optimizations* extend the local optimizations across branches and introduce a set of transformations aimed at optimizing loops.
- *Register allocation* associates registers with operands.
- *Processor-dependent optimizations* attempt to take advantage of specific architectural knowledge.

How Architecture Helps Compilers

How the Architect Can Help the Compiler Writer

Today, the complexity of a compiler does not come from translating simple statements like $A = B + C$. Most programs are *locally simple*, and simple translations work fine. Rather, complexity arises because programs are large and globally complex in their interactions, and because the structure of compilers means decisions are made one step at a time about which code sequence is best.

Compiler writers often are working under their own corollary of a basic principle in architecture: *make the frequent cases fast and the rare case correct*. That is, if we know which cases are frequent and which are rare, and if generating code for both is straightforward, then the quality of the code for the rare case may not be very important—but it must be correct!

Some instruction set properties help the compiler writer. These properties should not be thought of as hard-and-fast rules, but rather as guidelines that will make it easier to write a compiler that will generate efficient and correct code.

- *Provide regularity*—Whenever it makes sense, the three primary components of an instruction set—the operations, the data types, and the addressing modes—should be *orthogonal*. Two aspects of an architecture are said to be orthogonal if they are independent. For example, the operations and addressing modes are orthogonal if, for every operation to which one addressing mode can be applied, all addressing modes are applicable. This regularity helps simplify code generation and is particularly important when the decision about what code to generate is split into two passes in the compiler. A good counterexample of this property is restricting what registers can be used for a certain class of instructions. Compilers for special-purpose register architectures typically get stuck in this dilemma. This restriction can result in the compiler finding itself with lots of available registers, but none of the right kind!
- *Provide primitives, not solutions*—Special features that “match” a language construct or a kernel function are often unusable. Attempts to support high-level languages may work only with one language or do more or less than is required for a correct and efficient implementation of the language. An example of how such attempts have failed is given in [Section A.10](#).

- *Provide primitives, not solutions*—Special features that “match” a language construct or a kernel function are often unusable. Attempts to support high-level languages may work only with one language or do more or less than is required for a correct and efficient implementation of the language. An example of how such attempts have failed is given in [Section A.10](#).
- *Simplify trade-offs among alternatives*—One of the toughest jobs a compiler writer has is figuring out what instruction sequence will be best for every segment of code that arises. In earlier days, instruction counts or total code size might have been good metrics, but—as we saw in [Chapter 1](#)—this is no longer

A.8 Cross-Cutting Issues: The Role of Compilers ■ A-31

true. With caches and pipelining, the trade-offs have become very complex. Anything the designer can do to help the compiler writer understand the costs of alternative code sequences would help improve the code. One of the most difficult instances of complex trade-offs occurs in a register-memory architecture in deciding how many times a variable should be referenced before it is cheaper to load it into a register. This threshold is hard to compute and, in fact, may vary among models of the same architecture.

- *Provide instructions that bind the quantities known at compile time as constants*—A compiler writer hates the thought of the processor interpreting at runtime a value that was known at compile time. Good counterexamples of this principle include instructions that interpret values that were fixed at compile time. For instance, the VAX procedure call instruction (`call`) dynamically interprets a mask saying what registers to save on a call, but the mask is fixed at compile time (see [Section A.10](#)).

- The *stack* is used to allocate local variables. The stack is grown or shrunk on procedure call or return, respectively. Objects on the stack are addressed relative to the stack pointer and are primarily scalars (single variables) rather than arrays. The stack is used for activation records, *not* as a stack for evaluating expressions. Hence, values are almost never pushed or popped on the stack.
- The *heap* is used to allocate dynamic objects that do not adhere to a stack discipline. Objects in the heap are accessed with pointers and are typically not scalars.

Register allocation is much more effective for stack-allocated objects than for global variables, and register allocation is essentially impossible for heap-allocated objects because they are accessed with pointers. Global variables and some stack

Aims of RISC-V Architecture

Putting It All Together: The RISC-V Architecture



In this section we describe the load-store architecture called RISC-V. RISC-V is a freely licensed open standard, similar to many of the RISC architectures, and based on observations similar to those covered in the last sections. (In Section M.3 we discuss how and why these architectures became popular.) RISC-V builds on 30 years of experience with RISC architectures and “cleans up” most of the short-term inclusions and omissions, leading to an architecture that is easier and more efficient to implement. RISC-V provides both a 32-bit and a 64-bit instruction set, as well as a variety of extensions for features like floating point; these extensions can be added to either the 32-bit or 64-bit base instruction set. We discuss a 64-bit version of RISC-V, RV64, which is a superset of the 32-bit version RV32.

Reviewing our expectations from each section, for desktop and server applications:

- **Section A.2**—Use general-purpose registers with a load-store architecture.
- **Section A.3**—Support these addressing modes: displacement (with an address offset size of 12–16 bits), immediate (size 8–16 bits), and register indirect.
- **Section A.4**—Support these data sizes and types: 8-, 16-, 32-, and 64-bit integers and 64-bit IEEE 754 floating-point numbers.
- **Section A.5**—Support these simple instructions, because they will dominate the number of instructions executed: load, store, add, subtract, move register-register, and shift.
- **Section A.6**—Compare equal, compare not equal, compare less, branch (with a PC-relative address at least 8 bits long), jump, call, and return.
- **Section A.7**—Use fixed instruction encoding if interested in performance, and use variable instruction encoding if interested in code size. In some low-end, embedded applications, with small or only one-level caches, larger code size may have significant performance implications. ISAs that provide a compressed instruction set extension provide a way of addressing this difference.
- **Section A.8**—Provide at least 16, and preferably 32, general-purpose registers, be sure all addressing modes apply to all data transfer instructions, and aim for

RISC-V Instruction Set Organization

The RISC-V instruction set is organized as three base instruction sets that support 32-bit or 64-bit integers, and a variety of optional extensions to one of the base instruction sets. This allows RISC-V to be implemented for a wide range of potential applications from a small embedded processor with a minimal budget for logic and memory that likely costs \$1 or less, to high-end processor configurations with full support for floating point, vectors, and multiprocessor configurations. [Figure A.22](#) summarizes the three base instruction sets and the instruction set extensions with their basic functionality. For purposes of this text, we use RV64IMAFD (also known as RV64G, for short) in examples. RV32G is the 32-bit subset of the 64-bit architecture RV64G.

Name of base or extension	Functionality
RV32I	Base 32-bit integer instruction set with 32 registers
RV32E	Base 32-bit instruction set but with only 16 registers; intended for very low-end embedded applications
RV64I	Base 64-bit instruction set; all registers are 64-bits, and instructions to move 64-bit from/to the registers (LD and SD) are added
M	Adds integer multiply and divide instructions
A	Adds atomic instructions needed for concurrent processing; see Chapter 5
F	Adds single precision (32-bit) IEEE floating point, includes 32 32-bit floating point registers, instructions to load and store those registers and operate on them
D	Extends floating point to double precision, 64-bit, making the registers 64-bits, adding instructions to load, store, and operate on the registers
Q	Further extends floating point to add support for quad precision, adding 128-bit operations
L	Adds support for 64- and 128-bit decimal floating point for the IEEE standard
C	Defines a compressed version of the instruction set intended for small-memory-sized embedded applications. Defines 16-bit versions of common RV32I instructions
V	A future extension to support vector operations (see Chapter 4)
B	A future extension to support operations on bit fields
T	A future extension to support transactional memory
P	An extension to support packed SIMD instructions: see Chapter 4
RV128I	A future base instruction set providing a 128-bit address space

RV64G

Figure A.22 RISC-V has three base instructions sets (and a reserved spot for a future fourth); all the extensions extend one of the base instruction sets. An instruction set is thus named by the base name followed by the extensions. For example, RISC-V64IMAFD refers to the base 64-bit instruction set with extensions M, A, F, and D. For consistency of naming and software, this combination is given the abbreviated name: RV64G, and we use RV64G through most of this text.

Data Types for RISC-V

The data types are 8-bit bytes, 16-bit half words, 32-bit words, and 64-bit double-words for integer data and 32-bit single precision and 64-bit double precision for floating point. Half words were added because they are found in languages like C

Load and Store Instructions

Example instruction	Instruction name	Meaning
<code>ld x1, 80(x2)</code>	Load doubleword	$\text{Regs}[x1] \leftarrow \text{Mem}[80 + \text{Regs}[x2]]$
<code>lw x1, 60(x2)</code>	Load word	$\text{Regs}[x1] \leftarrow_{64} \text{Mem}[60 + \text{Regs}[x2]]_0^{32} \text{###}$ $\text{Mem}[60 + \text{Regs}[x2]]$
<code>lwu x1, 60(x2)</code>	Load word unsigned	$\text{Regs}[x1] \leftarrow_{64} 0^{32} \text{###} \text{Mem}[60 + \text{Regs}[x2]]$
<code>lb x1, 40(x3)</code>	Load byte	$\text{Regs}[x1] \leftarrow_{64} (\text{Mem}[40 + \text{Regs}[x3]]_0)^{56} \text{###}$ $\text{Mem}[40 + \text{Regs}[x3]]$
<code>lbu x1, 40(x3)</code>	Load byte unsigned	$\text{Regs}[x1] \leftarrow_{64} 0^{56} \text{###} \text{Mem}[40 + \text{Regs}[x3]]$
<code>lh x1, 40(x3)</code>	Load half word	$\text{Regs}[x1] \leftarrow_{64} (\text{Mem}[40 + \text{Regs}[x3]]_0)^{48} \text{###}$ $\text{Mem}[40 + \text{Regs}[x3]]$
<code>flw f0, 50(x3)</code>	Load FP single	$\text{Regs}[f0] \leftarrow_{64} \text{Mem}[50 + \text{Regs}[x3]] \text{###} 0^{32}$
<code>fld f0, 50(x2)</code>	Load FP double	$\text{Regs}[f0] \leftarrow_{64} \text{Mem}[50 + \text{Regs}[x2]]$
<code>sd x2, 400(x3)</code>	Store double	$\text{Mem}[400 + \text{Regs}[x3]] \leftarrow_{64} \text{Regs}[x2]$
<code>sw x3, 500(x4)</code>	Store word	$\text{Mem}[500 + \text{Regs}[x4]] \leftarrow_{32} \text{Regs}[x3]_{32..63}$
<code>fsw f0, 40(x3)</code>	Store FP single	$\text{Mem}[40 + \text{Regs}[x3]] \leftarrow_{32} \text{Regs}[f0]_{0..31}$
<code>fsd f0, 40(x3)</code>	Store FP double	$\text{Mem}[40 + \text{Regs}[x3]] \leftarrow_{64} \text{Regs}[f0]$
<code>sh x3, 502(x2)</code>	Store half	$\text{Mem}[502 + \text{Regs}[x2]] \leftarrow_{16} \text{Regs}[x3]_{48..63}$
<code>sb x2, 41(x3)</code>	Store byte	$\text{Mem}[41 + \text{Regs}[x3]] \leftarrow_8 \text{Regs}[x2]_{56..63}$

Figure A.25 The load and store instructions in RISC-V. Loads shorter than 64 bits are available in both sign-extended and zero-extended forms. All memory references use a single addressing mode. Of course, both loads and stores are available for all the data types shown. Because RV64G supports double precision floating point, all single precision floating point loads must be aligned in the FP register, which are 64-bits wide.

Understanding Address Conventions



To understand these figures we need to introduce a few additional extensions to our C description language used initially on page A-9:

- A subscript is appended to the symbol $\leftarrow$ whenever the length of the datum being transferred might not be clear. Thus, $\leftarrow_n$ means transfer an n -bit quantity. We use $x, y \leftarrow z$ to indicate that z should be transferred to x and y .
- A subscript is used to indicate selection of a bit from a field. Bits are labeled from the most-significant bit starting at 0. The subscript may be a single digit (e.g., $\text{Regs}[x4]_0$ yields the sign bit of $x4$) or a subrange (e.g., $\text{Regs}[x3]_{56..63}$ yields the least-significant byte of $x3$).
- The variable Mem , used as an array that stands for main memory, is indexed by a byte address and may transfer any number of bytes.
- A superscript is used to replicate a field (e.g., 0^{48} yields a field of zeros of length 48 bits).
- The symbol $\#\#$ is used to concatenate two fields and may appear on either side of a data transfer, and the symbols $\ll$ and $\gg$ shift the first operand left or right by the amount of the second operand.

Example instruction	Instruction name	Meaning
<code>add x1,x2,x3</code>	Add	$\text{Regs}[x1] \leftarrow \text{Regs}[x2] + \text{Regs}[x3]$
<code>addi x1,x2,3</code>	Add immediate unsigned	$\text{Regs}[x1] \leftarrow \text{Regs}[x2] + 3$
<code>lui x1,42</code>	Load upper immediate	$\text{Regs}[x1] \leftarrow 0^{32} \# \# 42 \# \# 0^{12}$
<code>sll x1,x2,5</code>	Shift left logical	$\text{Regs}[x1] \leftarrow \text{Regs}[x2] \ll 5$
<code>slt x1,x2,x3</code>	Set less than	$\text{if } (\text{Regs}[x2] < \text{Regs}[x3])$ $\text{Regs}[x1] \leftarrow 1 \text{ else } \text{Regs}[x1] \leftarrow 0$

Figure A.26 The basic ALU instructions in RISC-V are available both with register-register operands and with one immediate operand. LUI uses the U-format that employs the rs1 field as part of the immediate, yielding a 20-bit immediate.

Example instruction	Instruction name	Meaning
<code>jal x1,offset</code>	Jump and link	$\text{Regs}[x1] \leftarrow \text{PC} + 4; \text{PC} \leftarrow \text{PC} + (\text{offset} \ll 1)$
<code>jalr x1,x2,offset</code>	Jump and link register	$\text{Regs}[x1] \leftarrow \text{PC} + 4; \text{PC} \leftarrow \text{Regs}[x2] + \text{offset}$
<code>beq x3,x4,offset</code>	Branch equal zero	$\text{if } (\text{Regs}[x3] == \text{Regs}[x4]) \text{ PC} \leftarrow \text{PC} + (\text{offset} \ll 1)$
<code>bgt x3,x4,name</code>	Branch not equal zero	$\text{if } (\text{Regs}[x3] > \text{Regs}[x4]) \text{ PC} \leftarrow \text{PC} + (\text{offset} \ll 1)$

Figure A.27 Typical control flow instructions in RISC-V. All control instructions, except jumps to an address in a register, are PC-relative.

Example Assembly code for FP sum function

sum.s for RISC-V

```

1  .file "sum.c"
2  .text
3  .align 2
4  .globl sum
5  .type sum, @function
6  sum:
7      add sp, sp, -48
8      sd s0, 40(sp)
9      add s0, sp, 48
10     sw a0, -36(s0)
11     sd a1, -48(s0)
12     fsw fa2, -40(s0)
13     sw zero, -24(s0)
14     sw zero, -20(s0)
15     j .L2
16 .L3:
17     lw a5, -20(s0)
18     sll a5, a5, 2
19     ld a4, -48(s0)
20     add a5, a4, a5
21     flw fa4, 0(a5)
22     flw fa5, -40(s0)
23     fmul.s fa5, fa4, fa5
24     flw fa4, -24(s0)
25     fadd.s fa5, fa4, fa5
26     fsw fa5, -24(s0)
27     lw a5, -20(s0)
28     addw a5, a5, 1
29     sw a5, -20(s0)
30 .L2:
31     lw a4, -20(s0)
32     lw a5, -36(s0)
33     blt a4, a5, .L3
34     flw fa5, -24(s0)
35     fmv.s fa0, fa5
36     ld s0, 40(sp)
37     add sp, sp, 48
38     jr ra
39     .size sum, .-sum

```

```

1 float sum(int N, float X[], float a) {
2     int i;
3     float result = 0.0;
4     for (i = 0; i < N; ++i)
5         result += a * X[i];
6     return result;
7 }

```

2 or 3 operands
-20 (s0): Memory address

<https://riscv.org/>

Lecture 03 Instruction Set Principles

CSCE 513 Computer Architecture

Department of Computer Science and Engineering

Yonghong Yan

yanyh@cse.sc.edu

<http://cse.sc.edu/~yanyh>

Readings:
Appendix A of course textbook